

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт _____ информатики, математики и электроники
Факультет _____ информатики
Кафедра _____ программных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**«РАЗРАБОТКА ПАРАЛЛЕЛЬНОГО АЛГОРИТМА ДЛЯ
СТОХАСТИЧЕСКОГО МЕТОДА ДИНАМИЧЕСКОГО РАСШИРЕНИЯ
КЛАСТЕРА»**

по направлению подготовки 02.03.02

Фундаментальная информатика и информационные технологии
(уровень бакалавриата)

направленность (профиль) «Информационные технологии»

Студент _____ С.С. Сидоров

Руководитель ВКР,

доцент каф. ПС _____ Д.А. Попова-Коварцева

Самара 2020

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт _____ информатики, математики и электроники
Факультет _____ информатики
Кафедра _____ программных систем

УТВЕРЖДАЮ
Заведующий кафедрой

_____ А.Н. Коварцев
«___» _____ 2019 г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(бакалавр)

студенту _____ Сидорову Сидору Сидоровичу
группа _____ 6414-020302D

Тема работы: _____ Разработка параллельного алгоритма для стохастического
метода динамического расширения кластера

Цель работы: _____ Разработать параллельный алгоритм для стохастического
метода динамического расширения кластера

Структурные части работы (перечень вопросов, подлежащих разработке):

- 1) Провести анализ предметной области: обзор методов поиска оптимальной конформации атомного кластера
- 2) Разработать параллельную версию алгоритма формирования приближений конформаций атомов Морса на основе геометрически обоснованного метода
- 3) Реализовать разработанный алгоритм на языке C(C++)
- 4) Провести вычислительные эксперименты
- 5) Найти начальные конформации для кластеров, количество элементов которых превышает 270
- 6) Проанализировать полученные результаты

Научный руководитель
доцент, кафедра программных систем

_____ (Д.А. Попова-Коварцева)
«___» _____ 2019 г.

Задание принял к исполнению

_____ (С.С. Сидоров)
«___» _____ 2019 г.

РЕФЕРАТ

Пояснительная записка: 68с, 26 рисунков, 5 таблицы, 31 источников, 1 приложение.

Графическая часть: 17 слайдов презентации PowerPoint.

ГЛОБАЛЬНАЯ ОПТИМИЗАЦИЯ, МЕТОД МОНТЕ-КАРЛО, АТОМНЫЙ КЛАСТЕР, ПОТЕНЦИАЛ МОРСА, ГЕОМЕТРИЧЕСКИ ОБОСНОВАННЫЙ МЕТОД

Объектом исследования является процесс поиска глобально-оптимальных конформаций кластеров Морса.

Цель работы – разработать параллельный стохастический алгоритм формирования начальных конформаций кластеров Морса на основе геометрически обоснованного метода.

В процессе работы была изучена предметная область, разработан алгоритм построения начальных конформаций кластеров Морса на основе геометрически обоснованного метода. Работа алгоритма была проверена на кластерах размером 268-290 атомов.

Алгоритм разработан на языке C/C++ с применением MPI и дополнительных библиотек. Отладка произведена в Microsoft Visual Studio. Тестирование проведено на двух операционных системах: Windows и Linux. Подсчет оптимальных конформаций произведен при помощи вычислительных ресурсов кластера «Сергей Королев».

СОДЕРЖАНИЕ

Введение	6
1 Описание и анализ предметной области.....	8
1.1 Основные понятия и определения.....	8
1.1.1 Кластеры и кластерные соединения	8
1.1.2 Геометрическая структура кластера.....	8
1.1.3 «Магические числа».....	10
1.1.4 Коэффициент жесткости кластера.....	11
1.1.5 Потенциал Морса	13
1.1.6 Задача поиска оптимальной конформации кластера и глобальной оптимизации	14
1.1.7 Поверхность потенциальной энергии	16
1.1.8 Метод Монте-Карло	17
1.1.9 Генератор псевдослучайных чисел	18
1.2 Обзор существующих методов поиска оптимальной конформации.....	20
1.2.1 Генетические и эволюционные алгоритмы.....	21
1.2.2 Метод basin-hopping	22
1.2.3 Методы с конструированием ядра.....	24
1.3 Постановка задачи.....	26
2 Стохастический алгоритм для работы с кластерами Морса.....	27
2.1 Постановка задачи структурной оптимизации	27
2.2 Алгоритм динамического расширения кластера	30
2.3 Формирование плотной упаковки атомов.....	32
2.3.1 Определение координат центров атомов в секторе базового пятиугольника.....	33
2.3.2 Определение координат центров атомов в секторе деформированного пятиугольника.....	34
2.4 Параллельный алгоритм поиска оптимальных конформаций	36
2.5 Выводы по главе.....	37
3 Реализация разработанного алгоритма	39
3.1 Выбор и обоснование комплекса программных средств	39
3.2 Выбор и обоснование операционной системы	39

3.3	Выбор и обоснование языка программирования.....	40
3.4	Выбор и обоснование использования библиотеки PCG	40
3.5	Выбор и обоснование средства распараллеливания вычислений	41
3.6	Выбор и обоснование среды разработки	42
3.7	Дополнительное программное обеспечение.....	43
3.8	Выводы по главе.....	44
4	Вычислительные эксперименты.....	45
4.1	Сравнение последовательной и параллельной версии	50
4.2	Временная сложность алгоритма	52
4.3	Выводы по главе.....	54
	Заключение.....	55
	Список использованных источников	56
	ПРИЛОЖЕНИЕ А Код программы.....	60

ВВЕДЕНИЕ

Поиск оптимальных конформаций структур атомных кластеров является актуальной проблемой современной науки. Под оптимальной конформацией следует понимать конформацию, при которой достигается энергетический минимум для конкретного кластера.

Задача поиска оптимальных конформаций нашла свое отражение в разных областях науки. Например, кластеры Морса находят свое применение в задаче поиска оптимальной структуры фолдинга и докинга белка. Кластеры Морса представляют собой группу атомов или молекул, связанных между собой. Фолдингом белка называют процесс спонтанного разворачивания полипептидной цепи в уникальную пространственную структуру [1]. Так как механизмы, стоящие за сворачиванием белков на данный момент не изучены, экспериментальное определение трехмерной структуры белка часто очень сложно. В свою очередь моделирование процессов свёртывания и развёртывания белка поможет получить лучшее понимание причин возникновения болезней, вызываемых дефектными белками [2].

Кроме этого, согласно недавним исследованиям, атомные кластеры обладают люминесценцией в видимой области спектра. Это делает возможным применение кластеров в фотонике [3].

Главной задачей данного направления является обнаружение оптимальной геометрической структуры, которая бы соответствовала минимуму потенциальной энергии атомов, составляющих ее.

Для нахождения оптимальной конформации, которая бы обеспечила глобальный минимум потенциальной энергии, на данный момент существуют различные методы, так или иначе использующие дополнительную информацию о кластере. Одним из таких упрощений является геометрически обоснованный метод. Согласно ему, решетки расположения атомов кластера Морса в конформации тяготеют к сферической форме.

Методы, гарантирующие глобальную оптимальность конформации, оказались безрезультативными в этой сфере. Данная задача имеет экспоненциальный рост сложности в зависимости от размеров кластера. Стоит упомянуть другие эвристические методы, которые уже были задействованы в этой задаче ранее. Одним из таких методов является метод basin-hopping. Основная идея метода в том, что локальные экстремумы группируются в так называемые «бассейны», в каждом из которых локальные минимумы энергии расположены на столько близко, что за счёт случайных возмущений возможен постепенный переход от локальных экстремумов к глобальному [4].

Во время выполнения выпускной квалификационной работы необходимо разработать параллельный стохастический алгоритм построения начальных конформаций атомов кластера Морса на основе геометрически обоснованного метода.

1 Описание и анализ предметной области

1.1 Основные понятия и определения

1.1.1 Кластеры и кластерные соединения

Кластером называется система, состоящая из конечного числа однотипных частиц, обычно связанных [5]. В физике, под кластерами понимают коррелированные группы элементарных частиц. В химии – это одно из промежуточных состояний между одиночным атомом и твердым телом [6].

В том и другом случае, это частицы малого размера, свойства которого сильно отличаются от свойств макрообъекта.

Как уже было сказано выше, атомный или молекулярный кластер, состоящий из набора частиц, может проявлять свойства отличные от свойств одной молекулы или атома. Оценка его наиболее значимых свойств имеет непосредственное значение во многих областях, начиная с прогнозирования структуры белка до изучения влияния стратосферных облаков на разрушение озона [7]. Кроме того, правильное понимание свойств кластера имеет решающее значение в области нанотехнологий.

1.1.2 Геометрическая структура кластера

Задача формирования атомного кластера минимальной потенциальной энергии тесно связана с задачей плотной упаковки шаров в пространстве. Для кластеров наиболее важными факторами являются: сферичность формы конформации и достижимость как можно более плотной «упаковки» атомов [8].

Геометрическая структура кластера может быть неупорядоченной, икосаэдрической, полиикосаэдрической, тетраэдрической, декаэдрической, гранецентрированной кубической (ГЦК) и плотно упакованной. Плотно упакованный тип конформации является менее исследованным, чем все остальные. На рисунке 1 показаны основные конформации кластеров.

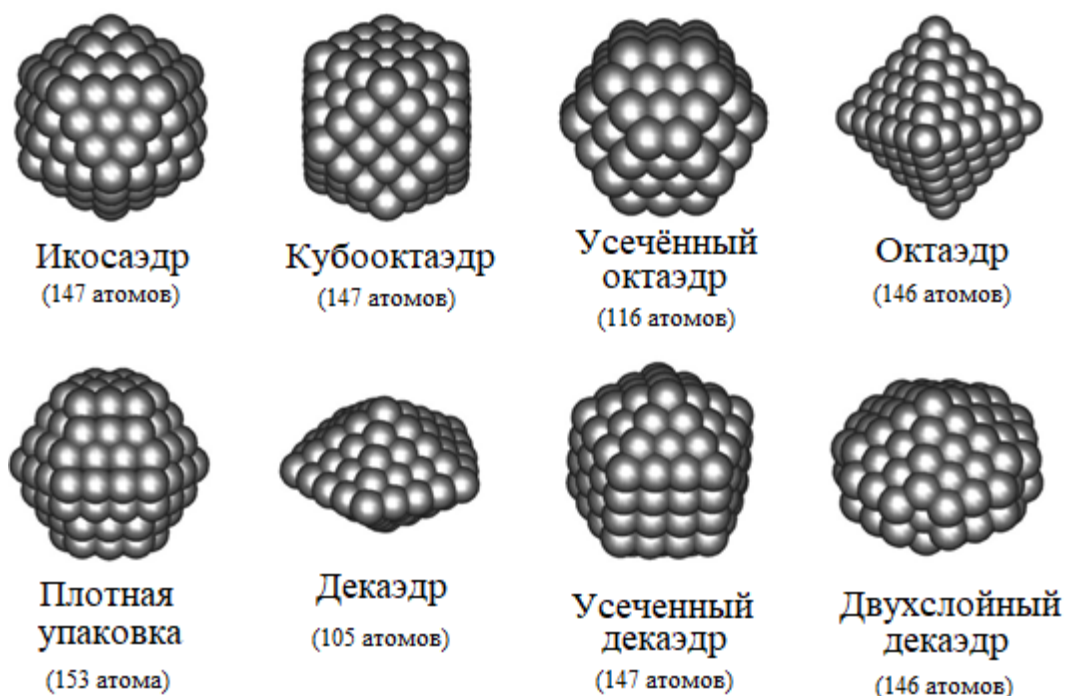


Рисунок 1 – Возможные конформации кластеров

Наибольшая плотность для жестких шаров достигается для гранецентрированной кубической решетки [8]. Структура, получаемая такой решеткой, является плотноупакованной. Атомы в ней уложены в узлах правильной треугольной сетки.

Основываясь на результатах, представленных в базе данных кластерных структур Кембриджского университета [9], можно заключить, что для больших кластерных структур с числом атомов $12 < N < 250$ глобальный оптимальный потенциал энергии получается при икосаэдрической и декаэдрической структуре [8].

Декаэдрические, икосаэдрические и гранецентрированные решетки являются хорошим приближением для поиска глобального минимума потенциальной энергии. Благодаря им можно заранее построить оптимальную упаковку атомов и уже в ней осуществлять поиск глобально-оптимальной конформации.

Также, в ходе анализа уже изученных кластеров Морса, было выявлено, что кластер можно развернуть вдоль оси z таким образом, что его атомы будут образовывать нечеткую слоистую структуру, т.е. кластер можно

представить в виде слоев. Кроме того, в проекции на плоскость xOy выделенные слои образуют икосаэдрическую нерешётчатую упаковку. На рисунке 2 представлена такая проекция. Две эти эвристики помогают более просто формировать кластер.

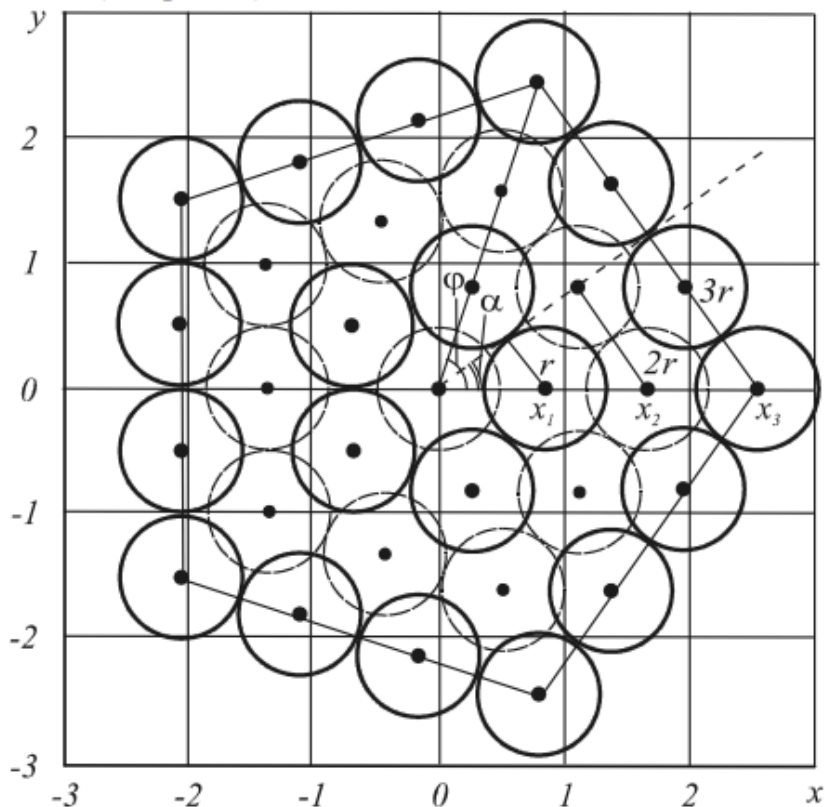


Рисунок 2 – Проекция нерешётчатой икосаэдрической упаковки на плоскость xOy

1.1.3 «Магические числа»

Понятие «Магических» чисел неразрывно связано с проблемой плотнейшей упаковки шаров в пространстве. Под ними понимается число атомов, при котором формируются ядра кластеров – конформации, являющиеся более выгодными по сравнению с конформациями содержащими иное число атомов. Магические числа предсказуемы и высчитываются по формуле (1).

$$N = \frac{1}{3} (10n^3 + 15n^2 + 11n + 3), \quad (1)$$

где n – число слоев вокруг центрального атома.

Оптимальные конформации, построенные для магических чисел являются хорошими ориентирами для формирования промежуточных кластеров. Они могут использоваться в качестве ядер для наращивания (присоединения к ним атомов) [8].

Было замечено, что конформации, расположенные между магическими числами, формируются путем переноса части его атомов на места центров шаров внешних пятиугольников икосаэдра. Так как построить икосаэдрическую сетку не составляет труда, это обстоятельство позволяет разрабатывать эвристический алгоритм формирования конформаций начальных приближений для кластеров с размерами между магическими числами [8].

1.1.4 Коэффициент жесткости кластера

Коэффициент жесткости кластера (ρ) – это число, которое показывает насколько шары, составляющие кластер, поддаются деформации. Иными словами, чем больше этот параметр, тем «тверже» атомы, тем сложнее составить из них шарообразную структуру. Кластер считается «жестким» если его коэффициент равен 14-ти. От жесткости кластера зависит ширина потенциальной ямы. Чем этот коэффициент выше, тем уже потенциальная яма и наоборот, чем он ниже, тем потенциальная яма шире. На рисунке 3 представлен график потенциала Морса для различных значений ρ . Каждая линия соответствует своему значению потенциала.

От параметра ρ зависит вид структуры составляющей кластер [10]. На рисунке 4 изображена фазовая диаграмма, показывающая изменение типа структуры с наименьшей энергией относительно N и ρ .

В случае, когда коэффициент ρ равен 14, оптимальной структура кластера будет достигаться в узлах икосаэдрической, гранецентрированной кубической и декаэдрической решеток.

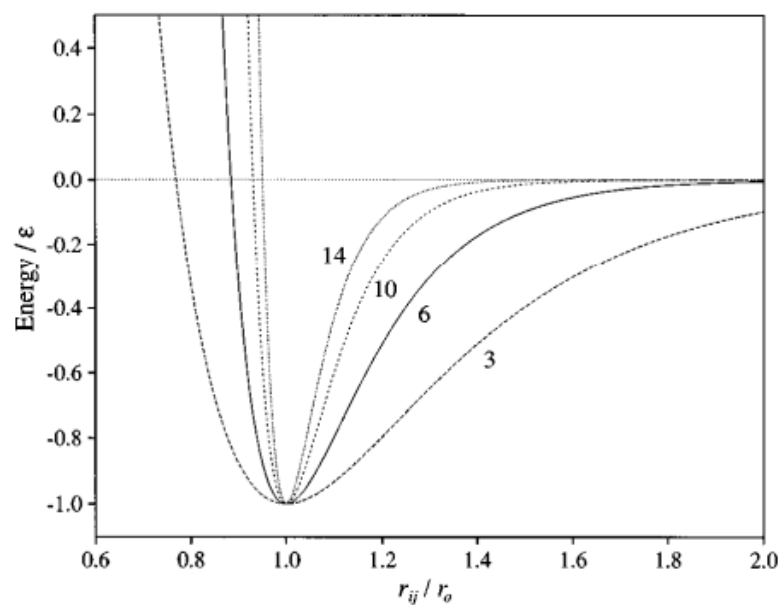


Рисунок 3 – График потенциала Морса для различных значений ρ

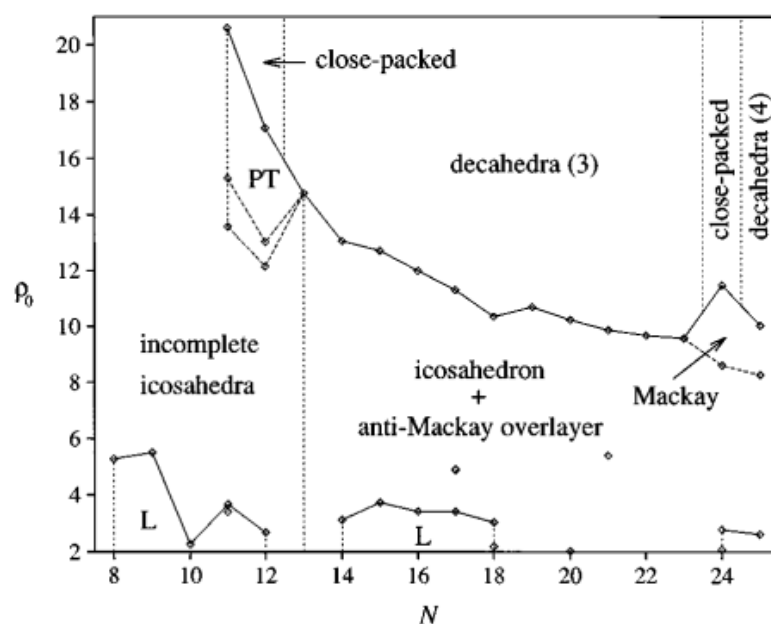


Рисунок 4 – Диаграмма изменения типа структуры кластера относительно N и ρ

Исходя из расчетов, размещенных в базе данных Кембриджского университета [9], потенциальная энергия кластера в зависимости от жесткости будет меняться следующим образом (таблица 1). Чем меньше жесткость кластера, тем плотнее можно упаковать атомы, следовательно, потенциальная энергия «мягких» кластеров будет меньше, чем «жестких».

Таблица 1 – Часть таблицы, размещенной в базе данных Кембриджского университета. Показывает значения потенциальной энергии кластеров при различных значениях коэффициента жёсткости

Количество атомов	Коэффициент жёсткости			
	$\rho=3$	$\rho=6$	$\rho=10$	$\rho=14$
7	-17.552961	-16.207580	-15.956512	-15.883113
8	-22.042901	-19.161862	-18.275118	-18.076248
8	-21.588831	-19.327420	-18.964638	-18.883688
9	-26.778449	-22.330837	-21.213531	-21.037957

1.1.5 Потенциал Морса

Потенциал Морса представляет собой удобную модель для потенциальной энергии межатомного взаимодействия. Как и другие модели описывающие межатомные взаимодействия (например потенциал Леннарда-Джонсона), потенциал Морса является приближением. Но, тем не менее, он достаточно реалистично описывает реальные физические процессы, происходящие при взаимодействии двух сферических молекул [11].

Потенциал Морса зависит всего от двух параметров: от жесткости ρ и от расстояния между центрами атомов r_{ij} . Он может быть записан в виде

$$v(X) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N e^{\rho(1-r_{ij})} (e^{\rho(1-r_{ij})} - 2), \quad (2)$$

где r_{ij} – расстояние между атомами i и j , ρ – коэффициент жесткости, $X = (x_1, \dots, x_N), x_i \in R^3$ – координаты центров атомов кластера.

Наиболее близкая ему модель – потенциал Леннарда-Джонсона. Как и потенциал Морса он описывает парное взаимодействие двух сферических молекул. График потенциала Леннарда-Джонсона изображен на рисунке 5.

Функция, описывающая потенциал Леннарда-Джонсона, выглядит следующим образом:

$$v(r) = LJ(r) = \varepsilon \left(\frac{r_0}{r} \right)^6 \left(\left(\frac{r_0}{r} \right)^6 - 2 \right), \quad (3)$$

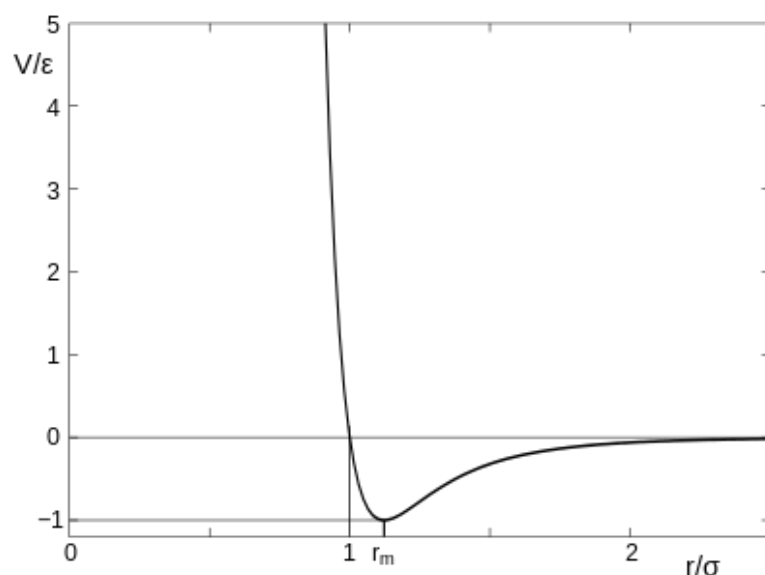


Рисунок 5 – Характерный вид потенциала Леннарда-Джонсона

где r_0 как правило равно единице.

Кластеры Леннарда-Джонсона на несколько порядков проще оптимизируются нежели кластеры Морса. Им свойственно большее структурное однообразие – практически все глобальные минимумы основаны на икосаэдре Маккея (рисунок 6).

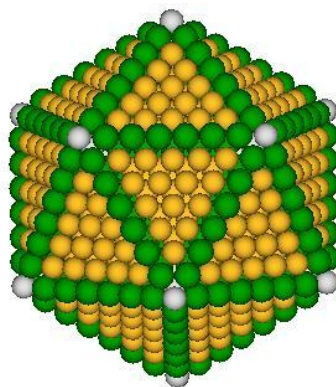


Рисунок 6 – Икосаэдр Маккея

1.1.6 Задача поиска оптимальной конформации кластера и глобальной оптимизации

В различных областях науки методы глобальной оптимизации находят свое применение. Они позволяют получить наилучший результат оптимизируемой функции и дают право утверждать, что лучшего результата невозможно достичь. Задачи, требующие оптимизации, возникают, например,

в теории управления, в проектировании, в анализе данных и многих других областях [12].

В общем виде, задача глобальной оптимизации выглядит как

$$F(x) \rightarrow \min_{x \in X}, \quad (4)$$

где многоэкстремальная целевая функция F является невыпуклой, $X \subset \mathbf{R}_k$ – допустимое множество задачи, $\mathbf{R}_k$ – k -мерное Евклидово пространство.

В общем случае нельзя гарантировать разрешимость задачи (что решение будет получено за конечное число шагов). Даже если задача является разрешимой, это не означает, что решение может быть получено, т.к. число шагов для достижения результата может быть чрезмерно большим [12].

Для неразрешимых задач существует несколько способов работы с ними. Первой способ – это сформулировать условия, которые бы приводили исходную задачу к разрешимому виду. Иными словами, сузить класс функций. Другой способ – отказаться от требования разрешимости задачи и получить только оценку глобального решения [12].

Все известные методы глобальной оптимизации можно разделить на две категории: детерминированные и стохастические. Первые – получают глобальное решение путем поиска по всем допустимым решениям. Они быстро теряют свою эффективность с ростом размерности задачи. Вторые, стохастические, оценивают значение целевой функции в случайных точках допустимого множества. В отличие от детерминированных, они не гарантируют успех, но класс задач, на которых они могут применяться, значительно шире [12]. Среди стохастических методов можно назвать: метод Монте-Карло, методы группировок, метод деления пополам, метод моделирования обжига, эволюционные алгоритмы и другие [12].

Для задачи оптимизации кластеров Морса был предложен двухэтапный алгоритм глобальной оптимизации. На первом этапе алгоритма формируются эффективные начальные приближения, на втором – с помощью непрерывных методов локальной оптимизации находятся точные решения, в которых содержится глобальный экстремум [4].

Пусть

$$X_1^{(N)}, X_2^{(N)}, \dots, X_k^{(N)}, \dots \quad (5)$$

упорядоченные по мере возрастания потенциальной энергии конформации кластеров Морса из N атомов.

$$v(X_1^{(N)}) \leq v(X_2^{(N)}) \leq \dots \leq v(X_k^{(N)}) \leq \dots \quad (6)$$

В данном случае $X_1^{(N)}$ – наилучшая конформация из N атомов. Присоединение к каждой из этих конформаций еще одного атома породит кластер Морса наименьшей потенциальной энергии. Тогда, шаг за шагом можно построить кластеры Морса для любого количества атомов.

1.1.7 Поверхность потенциальной энергии

Поверхность потенциальной энергии – это многомерная функция, показывающая зависимость энергии молекулы от ее конформации [13]. Данная функция имеет экстремумы, соответствующие минимумам и максимумам потенциальной энергии молекулы, при ее перемещении по доступному конформационному пространству. Самый «глубокий» минимум называется глобальным, а все остальные – локальными.

На рисунке 7 изображен условный график сечения потенциальной поверхности молекулярной системы.

Задача оптимизации кластеров Морса осложняется тем, что при больших значениях r , поверхность потенциальной энергии имеет «воронки», что означает существование конформаций с конкурентно малым значением энергии.

Для того, чтобы удобнее изображать поверхность потенциальной энергии, в случае ограниченного числа возможных конформаций, строится так называемый граф разбиения (рисунок 8)

Так как задача нахождения глобального минимума атомного кластера представляет собой оптимизацию невыпуклой функции с большим количеством локальных минимумов, исследование поверхности



Рисунок 7 – Сечение поверхности потенциальной энергии молекулярной системы

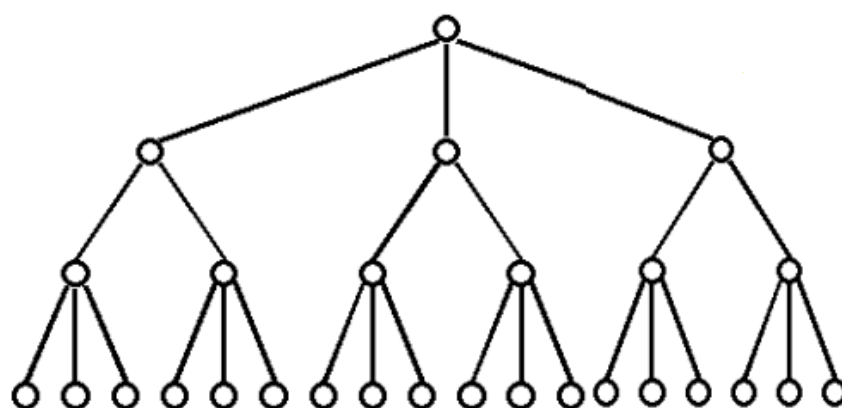


Рисунок 8 – Граф разбиения поверхности потенциальной энергии

потенциальной энергии является важным, и позволяет смотреть на задачу в более общем виде и применять к ней классические методы.

1.1.8 Метод Монте-Карло

Метод Монте-Карло представляет собой широкий класс вычислительных алгоритмов, которые полагаются на многократную случайную выборку для получения численного результата [14]. Данный метод является стохастическим и применяется для детерминированных задач, таких как: решение интегральных уравнений, краевых задач и т.д. Алгоритм

включает операции со случайными числами, в следствии чего результат вычисления тоже является случайным. Метод Монте-Карло часто используется в физических и математических задачах, где трудно или невозможно использовать другие подходы. Основная идея метода в том, что результаты рассчитываются на основе повторной случайной выборки и статистического анализа.

В решаемой задаче поиска глобально-оптимальной конформации кластера Морса, метод Монте-Карло используется для алгоритма его динамического расширения. Суть динамического расширения заключается в равномерном наращивании ядра кластера до заданных размеров.

Вообще говоря, любой метод, основанный на методе Монте-Карло делится на три этапа:

- 1 Определяется модель или алгоритм, который наиболее точно описывает систему.

- 2 Модель многократно применяется с использованием генератора случайных чисел. Далее она записывается в форме уравнения, выражающего соотношение между входными данными и выходными параметрами. Значения, отобранные в качестве входных данных, получают исходя из соответствующих распределений вероятностей.

- 3 С помощью компьютера многократно используют модель с различными входными параметрами и получают выходные данные. Они могут быть обработаны с помощью статистических методов для получения оценки среднего, стандартного отклонения и доверительных интервалов.

1.1.9 Генератор псевдослучайных чисел

Генератор псевдослучайных чисел (ГПСЧ) — алгоритм, порождающий последовательность чисел, элементы которой почти независимы друг от друга и подчиняются заданному распределению (обычно равномерному) [15]. Последовательность, получаемая ГПСЧ не является действительно случайной, так как она полностью зависит от начального значения.

Еще в середине двадцатого века А.Н. Колмогоров доказал, что невозможно построить алгоритм, генерирующий последовательность случайных натуральных чисел, поскольку любой компьютер работает с дискретной последовательностью чисел [16].

ГСПЧ являются определяющими в таких приложениях как моделирования, например, в упомянутом выше методе Монте-Карло. От качества используемых ГПСЧ зависит качество получаемых результатов.

На практике, выходные данные многих распространенных ГПСЧ демонстрируют артефакты, которые приводят к тому, что они не проходят статистические тесты. К ним относятся:

- Более короткие периоды для некоторых начальных состояний;
- Отсутствие равномерности распределения для большого количества генерируемых чисел;
- Корреляция между генерируемыми числами;
- Расстояние между определенными значениями распределены отлично от тех, которые получаются при случайной генерации.

Самым первым подходом к генерации ПСЧ был предложен Фон Нейманом в 1946 году и известен как метод средних квадратов. Алгоритм предлагал взять некоторое число, затем возвести его в квадрат, выбрать из середины числа четырехзначное число, которое является случайным; затем повторить операцию, подав на вход это число, т.е. снова возвести в квадрат и выбрать из середины четырехзначное число [15].

Недостатком данного метода является ограниченность множества ПСЧ из-за того, что последовательность закикливается – $5000^2 = 25000000,0^2 = 0, \dots$ и т.д.

Наиболее популярным на данный момент является линейный конгруэнтный метод. Он входит в стандартные библиотеки различных компиляторов и не обладает криптографической стойкостью [16].

Линейный конгруэнтный метод был предложен Д. Г. Лемером в 1949 году. Суть метода заключается в вычислении последовательности случайных чисел X_n полагая

$$X_{n+1} = (aX_n + c) \bmod m, \quad (13)$$

где m – натуральное число, относительно которого вычисляется остаток от деления, a – множитель ($0 \leq a < m$), c – приращение ($0 \leq c < m$), X_0 – начальное значение ($0 \leq X_0 < m$).

Эта последовательность называется линейной конгруэнтной последовательностью.

В большинстве популярных компиляторов, в которых используется данный метод, период ГПСЧ составляет от 2^{32} - 2^{64} , чего недостаточно при использовании в методе Монте-Карло.

В данной работе предлагается использовать так называемый PCG (Permuted congruential generator), который является модернизацией линейного конгруэнтного метода, и предлагающий лучшие статистические свойства за счет перестановок [17]. Кроме этого, он обладает возможностями расширения периода генерации до 2^{524352} , и параллельной работы на 2^{63} потоках [18]. Основная идея PCG состоит в том, чтобы числа «среднего качества», получаемые при генерации быстрыми ГСПЧ, пропускать через эффективную функцию перестановки (хеш-функцию) и улучшать тем самым выходные данные.

1.2 Обзор существующих методов поиска оптимальной конформации

Как уже было отмечено выше, задача поиска оптимальных конформаций имеет высокий рост сложности относительно растущего количества атомов. В связи с этим, поиск глобального минимума для кластеров больших размеров остается актуальной проблемой. Прямые методы глобальной оптимизации теряют свою актуальность для функций с числом переменных 20-30, в то время как в задаче формирования оптимального кластера участвуют $3N$ переменных. Для того чтобы как-либо

упростить задачу применяют различные эвристики, которые легли в основу имеющихся алгоритмов.

Среди существующих на данный момент алгоритмов поиска оптимальных конформаций следует выделить следующие: генетические и эволюционные алгоритмы (метод симуляции отжига, популяционный алгоритм); алгоритмы, основанные на идее трансформирования поверхности потенциальной энергии (алгоритм случайного спуска, алгоритм перемещения по воронкам); алгоритмы поиска на динамической сетке (метод с конструированием ядра, метод с операцией поворота).

1.2.1 Генетические и эволюционные алгоритмы

Общая идея эволюционных алгоритмов заключается в использовании механизмов, аналогичных механизмам естественного отбора в природе [19].

В классическом виде эволюционный алгоритм требует описание задачи таким образом, чтобы с сущностями можно было производить операции селекции, т.е. формируется вектор генов, которыми они обмениваются при кроссовере. Кроме этого, должна быть так называемая функция приспособленности, при помощи которой оценивается «приспособленность» некоторого генотипа.

Общая схема эволюционных алгоритмов обычно состоит из следующих шагов:

- 1 Некоторым, обычно случайным, методом задается множество генотипов начальной популяции.

- 2 Для каждого генома вычисляется значение функции приспособленности, с учетом которых к геному применяются операции селекции и мутации. Таким образом формируется новое поколение.

- 3 Если решение не найдено, то это потомство используется как родительское для создания следующего поколения.

Основной проблемой применения этого класса алгоритмов к задаче поиска оптимальных конформаций является его вычислительная сложность.

Однако, для понижения сложности можно использовать эволюционные алгоритмы с быстрой сходимостью, например, метод дифференциальной эволюции. Этот метод был представлен в работе [20], а его адаптация по задаче оптимизации кластера в работе [21].

1.2.2 Метод basin-hopping

Basin-hopping это семейство алгоритмов, основанных на идее о том, что локальные экстремумы потенциальной функции группируются в так называемые «бассейны». В каждом из таких бассейнов воронки локальных минимумов расположены настолько близко, что переход из одной воронки в другую возможен в результате случайных возмущений координат атомов.

Этот алгоритм особенно себя зарекомендовал в задаче глобальной оптимизации с воронкообразным энергетическим ландшафтом, какой является поиск энергетически минимальной структуры молекул [22].

Basin-hopping был впервые описан D. J. Wales и J. P. K. Doye и послужил значительным прорывом в оптимизации кластеров. Он также послужил началом исследования возможностей подходов, трансформирующих ландшафт потенциальной энергии.

Метод Basin-hopping основывается на структурной особенности целевой функции, заключающейся в том, что среди точек, расположенных в окрестностях локального минимума высока вероятность найти минимумы с более низким значением функции потенциальной энергии. Последовательно переходя из одних локальных минимумов в другие, более выгодные, можно постепенно, шаг за шагом, попасть в точку, являющуюся глобальным минимумом.

Основной характеристикой этого метода можно считать функцию поиска окрестностей, которая каждому локальному оптимуму ставит в соответствие точки близлежащих минимумов. Тогда можно определить направленный граф и сформулировать задачу глобальной оптимизации как задачу умного исследования графа.

Метод basin-hopping лег в основу многих других, одним из которых является так называемый метод funnel-hopping представленный в работе [23]. Общую схему алгоритма можно представить в виде следующих этапов:

- Выбор атомов с наибольшим вкладом энергии. Для каждого атома подсчитывается его потенциальный вклад в общую энергию.
- Построение графа окрестностей. Вершинами в графе являются атомы, а ребрами – связь между ними. Два атома считаются связанными если расстояние между ними меньше r_0 .
- Выделение в графе треугольников. В графе выделяются клики мощностью 3.
- Построение сетки. Для каждого треугольника ищутся две вершины правильного тетраэдра, которые включаются в структуру, именуемую динамической сеткой.
- Стягивание сетки. Для каждой точки запускается алгоритм локальной минимизации энергии кластера с N фиксированными положениями атомов и одним переменным положением атома, которое определяется точкой динамической сетки.
- Применение стратегии комбинаторного поиска для нахождения комбинации атомов с большим вкладом в общую энергию для перемещения их в места, где их вклад уменьшается.

В сущности, указанный подход является разновидностью метода ВН, где стратегия поиска окрестности определяется k операциями обмена между свободными положениями и положениями атомов с большим энергетическим вкладом.

В работе [23] было сказано об успешных результатах работы алгоритма на кластерах Леннарда-Джонса и Морса. Результаты подтверждались известными конформациями, полученными другими алгоритмами.

1.2.3 Методы с конструированием ядра

Методы, основанные на теории basin-hopping, теряют свою эффективность на кластерах больших размеров. Для кластеров с числом атомов свыше 561 они теряют свою эффективность или вовсе становятся неприменимыми. Для того чтобы ускорить процесс оптимизации были предложены стратегии, основанные на конструировании динамической сетки.

Алгоритмы этого класса осуществляют поиск по так называемой динамической сетке. Атомы с большим вкладом в энергию системы перемещаются в более выгодные позиции, тем самым приводя кластер к оптимальному состоянию. Среди алгоритмов поиска по динамической сетке (dynamic lattice searching, DLS) можно выделить следующие: включает метод поиска с конструированием ядра (DLS with constructed core, DLSc), метод с операцией поворота (DLS with rotation operation, DLS-RO), метод с двухэтапной локальной оптимизацией (DLS with two-phase local minimization method).

Алгоритм DLS начинается с генерации начальной структуры и выполнения процедур конструирования сетки. После этого формируются и исследуются позиции динамической сетки для поиска оптимальных решений. Начальная конформация получается путем применения локальной минимизации к случайно сгенерированной структуре. Процедуры конструирования сетки и поиска на сетке повторяются до тех пор, пока не будет найден новый минимум с лучшим значением энергии.

Из анализа классического метода DLS стало очевидным то, что в ходе оптимизации больший акцент делается на внешних атомах. В случае, если атомы ядра в стартовой конформации генерируются неудачно, задача поиска глобального минимума становится крайне сложной и трудоемкой. Именно по этой причине был предложен метод DLSc [24], в котором предлагается использование ядра декаэдрической или икосаэдрической структуры для существенного ускорения вычислений.

Эффективным оказалось применение этого подхода вместе с техникой поворота. Данный метод был применен для оптимизации кластеров Леннарда-Джонса размером свыше 500 атомов, кластеров Морса и кластеров некоторых металлов: серебра и алюминия.

Оператор поворота – это эвристический оператор, в ходе применения которого часть кластера поворачивается на случайный угол вокруг заданной или случайной оси, что позволяет добиться достаточной структурной трансформации кластера для выхода из локального минимума. Путем применения этого оператора декаэдрические структуры могут быть преобразованы в икосаэдрические. На рисунке 9 изображено такое преобразование.

Алгоритм поиска по динамической сетке с конструированием ядра и операцией поворота (DLSc-RO) можно описать следующими этапами:

- Генерация начальной структуры с ядром. При этом размер ядра определяется размером кластера. В качестве ядра берется декаэдр.
- Применение к стартовой структуре процедуры DLS для получения структуры с фрагментами декаэдра.

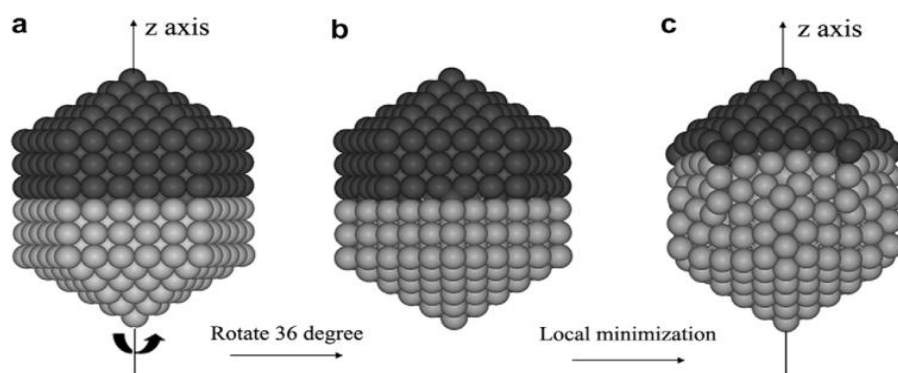


Рисунок 9 – Структурное преобразование декаэдра в икосаэдр путём поворота на 36° и процедуры локальной оптимизации

- Применение оператора поворота для получения икосаэдрической структуры.
- Дальнейшее применение процедуры DLS для улучшения икосаэдрической структуры.

Причем при такой стратегии поиска глобальный минимум может быть найден как среди декаэдрических структур, так и среди икосаэдрических.

1.3 Постановка задачи

В рамках выпускной квалификационной работы рассматривается задача поиска глобально-оптимальных конформаций кластеров Морса. Для решения данной задачи используется стохастический алгоритм, основанный на использовании заранее сгенерированной решетки, в узлах которой размещаются атомы кластера. Также, учитывая вычислительную сложность задачи, требуется разработать параллельную версию алгоритма, которая поможет ускорить поиск оптимальных конформаций.

При помощи алгоритма найти оптимальные конформации кластеров больших размеров, а именно, имеющих размер 270 атомов и более. Полученные результаты проанализировать и сделать выводы.

2 Стохастический алгоритм для работы с кластерами Морса

2.1 Постановка задачи структурной оптимизации

Как уже было сказано выше, для нахождения оптимальных конформаций, на данный момент используются различные методы, так или иначе использующие эвристики, упрощающие задачу. В данной работе используется геометрически-обоснованный метод, суть которого заключается в том, чтобы располагать атомы кластера в известных местах, определяемых типом решетки. Также, здесь рассматривается поиск оптимальных конфигураций для кластеров, коэффициент жесткости которых равен 14, т.е. атомы могут лишь касаться друг друга.

Используя заранее сформированную решетку, можно существенно упростить поставленную задачу поиска $3N$ координат атомов, сведя ее к задаче структурной оптимизации – выбору N наиболее выгодных атомов из M доступных.

Как уже было сказано в 1.1.3 главе, кластерам Морса с коэффициентом жесткости 14 свойственно наследовать некоторую часть атомов, называемую ядром. Однако, стоит заметить, что в общем случае каждый последующий кластер не всегда повторяет своего предшественника, иногда кардинально меняя тип решетки. Тем не менее, анализируя оптимальные конформации кластеров меньших размеров, в большинстве, можно выделить устойчивую группу атомов.

Анализ опубликованных в Кембриджской базе данных [9] оптимальных кластеров показал, что выбор атомов кластера происходит из ограниченной области икосаэдрической решетки. Однако, в определенные моменты происходят качественные изменения геометрической структуры глобально-оптимальных конформаций. В работе [25] было показано, что такие изменения возникают, например, на числе атомов равном 201 (рисунок 10). Вместо центрированной икосаэдрической решетки, там оптимальной становится смещенная икосаэдрическая решетка.

Слой	$N=192$	$N=196$	$N=200$	$N=201$	$N=203$	$N=209$
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						

Рисунок 10 – Таблица оптимальных конформаций для $N=192, 196, 200, 201, 203, 209$

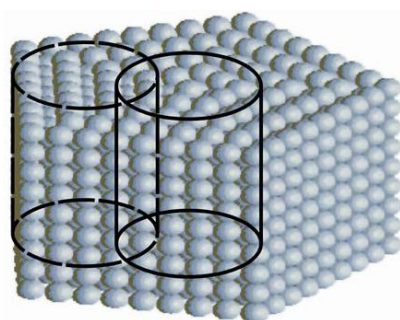


Рисунок 11 – Икосаэдрическая решетка

Для того чтобы получить заготовку для поиска начальных конформаций предлагается «вырезать» правильными геометрическими фигурами, например, цилиндром области центрированной и смещенной икосаэдрической решетки. Данная операция проиллюстрирована на рисунке 11.

Введем множество $X_{seq} = \{X_1, X_2, \dots, X_N\}$ как подмножество икосаэдрической сетки, на котором ищется глобально-оптимальная конформация. Определим множество $X_{cor} = \{\tilde{X}_{i_1}, \tilde{X}_{i_2}, \dots, \tilde{X}_{i_L}\}, \forall k \tilde{X}_k \in X_{seq}, k = 1, \dots, M$ как группу атомов кластера, образующая ядро. Множество $X_{var} = X_{seq} \setminus X_{cor}$ – свободные вершины решетки.

Сформируем граф G , учитывающий связь атомов между собой. Построим матрицу инцидентности графа G следующим образом:

$$A_{seq} = \|a_{ij}\|, \quad i, j = 1, \dots, M; \quad a_{ij} = \begin{cases} 1, & \text{если } \|X_i - X_j\| \leq r_{min} \\ 0, & \text{иначе} \end{cases} \quad (7)$$

где r_{min} – минимальное расстояние между атомами, которое фиксирует связь. Матрица инцидентности фактически определяет граф поиска $G(X_{seq}, U, A_{seq})$. Введем подграф $H(X', U', A')$ графа G . Причем $|A'| = N$, т.е. количество вершин графа равно размеру оптимизируемой конформации. Таких графов будет большое, но перечислимое количество.

Определим функцию выбора оптимального графа $\varphi(H_\alpha)$, обозначающую полезность данной конфигурации кластера. В качестве такой функции можно использовать потенциальную энергию парных взаимодействий атомов или сумму степеней вершин графа

$$\varphi(H_\alpha) = \sum_{X'_i \in X'} \deg(X'_i). \quad (8)$$

Тогда задачу структурной оптимизации можно сформулировать следующим образом:

$$\begin{cases} \min(\max) \varphi(H_\alpha), \\ H_\alpha \subset G, \\ |X'| = N. \end{cases} \quad (9)$$

Данная задача относится к классу задач глобальной оптимизации, но при этом имеет меньшую сложность, чем задача оптимизации (2), так как она содержит меньшее количество переменных и является задачей дискретной, а не параметрической оптимизации.

Общая блок-схема алгоритма изображена на рисунке 12.

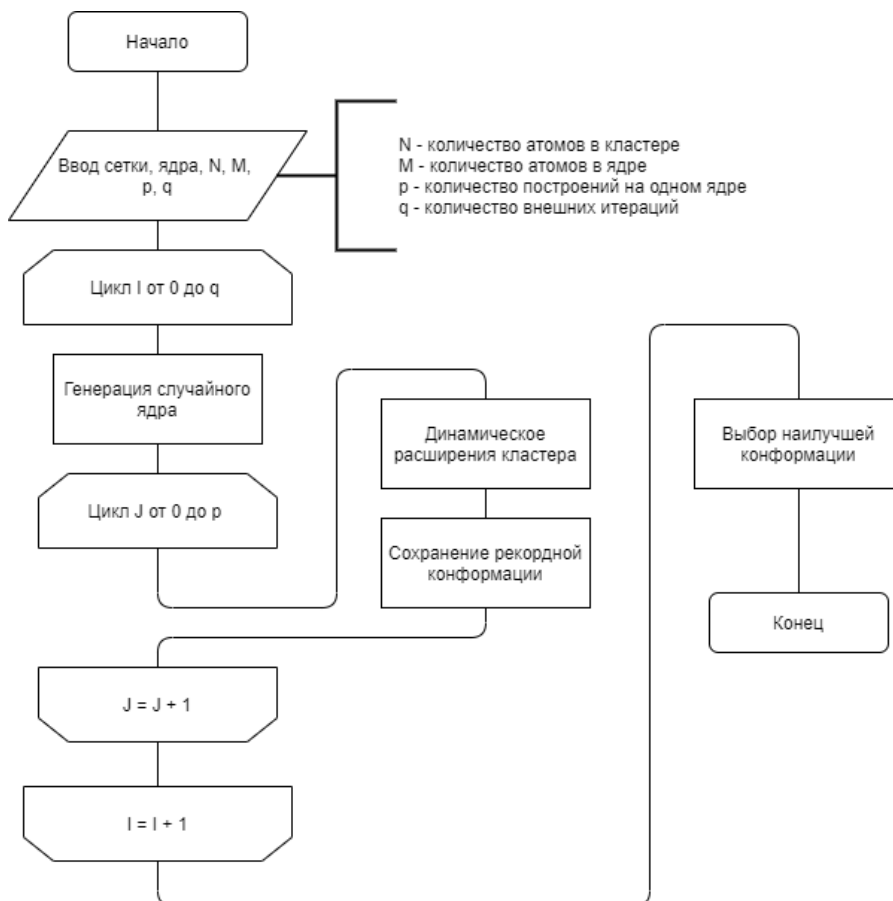


Рисунок 12 – Блок-схема разрабатываемого алгоритма

2.2 Алгоритм динамического расширения кластера

Алгоритм динамического расширения кластера, который был предложен Коварцевым А.Н. в работе [26], состоит из следующих шагов:

Шаг 1. Формируется матрица смежности расположения реальных или фантомных атомов в позициях их рационального расположения в узлах поисковой решетки. При этом два атома считаются связанными сильной потенциальной связью, если расстояние между атомами $\rho < 1.08$. Граф связей описывается в системе типологически – описанием связей всех вершин графа. Для этого используется массив структур:

$$Gr = \begin{cases} from - \text{номер текущей вершины;} \\ to - \text{список смежных вершин;} \\ deg - \text{степень вершины.} \end{cases} \quad (10)$$

Фактически каждая позиция решетки описывается с указанием все смежных узлов решетки.

Шаг 2. В работе [25] было показано, что кластеры больших размеров наследуют типовые конфигурации внутренних слоев кластера меньших размеров. Наследуемую часть атомов назовем ядром кластера. Ядро кластера условно сворачивается в одну вершину с номером 0. При построении кластера до нужных размеров нам теперь нужны связи ядра кластера (вершины 0) со свободными (не занятыми) вершинами поисковой решетки.

Далее формируются матрица смежности свободных вершин графа Gr_0 и список вершин ядра кластера $Top0$. Вычисляется потенциал ядра кластера;

Шаг 3. Основная идея предлагаемого алгоритма сводится к динамическому пошаговому расширению кластера, путем присоединения наиболее перспективных узлов решетки (атомов кластера) из числа свободных вершин.

Шаг 3.1. Пусть $Top0$ – список вершин ядра кластера; $Top1$ – список присоединенных вершин на данном шаге алгоритма. Остальные считаются свободными. Тогда множество текущих вершин кластера: $Top11 = Top0 \cup Top1$ и недостроенный кластер имеет вид $XS = \bigcup_{i \in Top11} X_i$. Его потенциальная энергия вычисляется по формуле $v11 = v(XS)$.

Шаг 3.2. Далее для всех свободных вершин решетки подсчитывается их возможный вклад в энергию кластера XS , если таковую $X_j \notin Top11$ присоединить к кластеру. На основании чего можно построить матрицу эффективности свободных вершин

$$TopG = \begin{pmatrix} T_1 & T_2 & \dots & T_{n1} \\ v(X_1) & v(X_2) & \dots & v(X_{n1}) \\ v(XS \cup X_1) & v(XS \cup X_2) & \dots & v(XS \cup X_{n1}) \end{pmatrix}, \quad (11)$$

где T_j – номер свободной вершины; $v(X_j)$ – потенциальный вклад свободной вершины в общую энергию кластера; $v(XSUX_j)$ – общая энергия кластера с вершиной T_j .

Шаг 3.3. Матрица TopG сортируется по второму показателю и получается новый список свободных вершин, упорядоченный по критерию вклада потенциальной энергии:

$$TopS = \begin{pmatrix} \widetilde{T}_1 & \widetilde{T}_2 & \dots & \widetilde{T}_{n1} \\ v(\widetilde{X}_1) & v(\widetilde{X}_2) & \dots & v(\widetilde{X}_{n1}) \\ v(XS \cup \widetilde{X}_1) & v(XS \cup \widetilde{X}_2) & \dots & v(XS \cup \widetilde{X}_{n1}) \end{pmatrix}, \quad (12)$$

Матрица TopS усекается до первых m членов, удовлетворяющих условию:

$$|v(\widetilde{X}_j) - v(\widetilde{X}_1)| \leq 0.02$$

В результате получается матрица TopMax, состоящая из лучших претендентов на присоединение к кластеру.

Шаг 3.4. Далее возможны два варианта:

- 1 Если $\|TopMax\|=1$, то выбирается первая вершина списка TopS.
- 2 Если $\|TopMax\|>1$, тогда равновероятно разыгрывается одна из вершин множества TopMax.

Далее «выигрышная» вершина Tr_j присоединяется к кластеру, а из множества свободных вершин она исключается.

Процесс динамического пошагового расширения кластера (шаги 1-3) повторяются многократно (итерационный процесс продолжается до тех пор, пока кластер не приобретет нужные размеры.). Число попыток фиксировано. Исходная ситуация одинакова для всех попыток. Рекордная конформация запоминается. Сложность алгоритма равна N .

2.3 Формирование плотной упаковки атомов

Как уже было упомянуто выше, использование геометрически обоснованного метода для поиска оптимальных конформаций кластеров

Морса предполагает наличие предварительно созданной икосаэдрической решетки (рисунок 13).

Рассмотрим алгоритмы формирования пятиугольных конфигураций атомов, предложенные в работе [6].

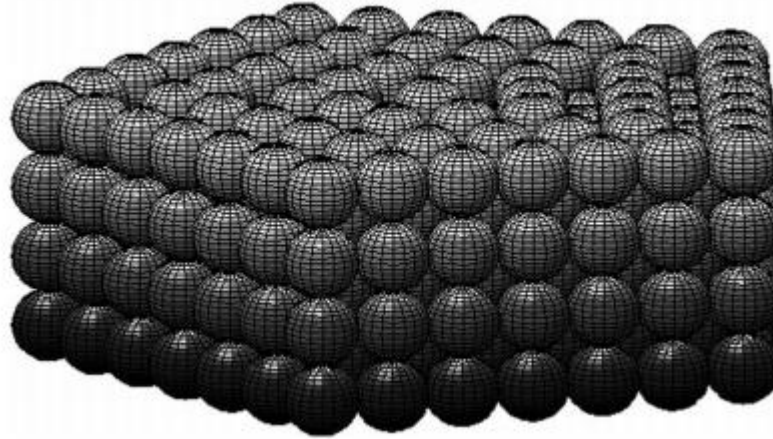


Рисунок 13 – Икосаэдрическая нерешетчатая упаковка шаров
(4 четных слоя и 3 нечетных)

2.3.1 Определение координат центров атомов в секторе базового пятиугольника

Рассмотрим алгоритм построения базовой конформации (рисунок 10). Каждый из слоев сетки строится следующим образом.

Пусть $\alpha = 2\pi/5$, $\beta = 2\pi/10$ и на каждой стороне размещается m шаров. Координату x_{m1} можно вычислить по следующей формуле

$$x_{m1} = mr / \sin(\pi/5). \quad (13)$$

а угол луча, проходящего через центр второго шара, по формуле

$$\gamma_k = \arctg \frac{a(k-1)}{x_{m1} - (k-1)b}, \quad k = 1, \dots, m, \quad (14)$$

где $a = 2r \sin \beta$, $b = 2r \cos \beta$.

Координаты центров атомов будут расположены на пересечении лучей под углами γ_k и прямой, проходящей через центры 1-го и m -го шара.

Конечная формула для вычисления координат центров атомов, лежащих на стороне пятиугольника, представима в виде

$$\begin{cases} x_{mk} = \frac{x_{m1} \operatorname{tg}(\pi - \beta)}{\operatorname{tg}(\pi - \beta) - \operatorname{tg} \gamma_k} \\ y_{mk} = \operatorname{tg}(\pi - \beta)(x_{mk} - x_{m1}) \end{cases} \quad (15)$$

Координаты центров шаров остальных четырёх секторов вычисляются за счет четырёхкратного вращения сектора на угол α .

На рисунке (14) проиллюстрирована процедура вычисления координат центров атомов в секторе базового пятиугольника.

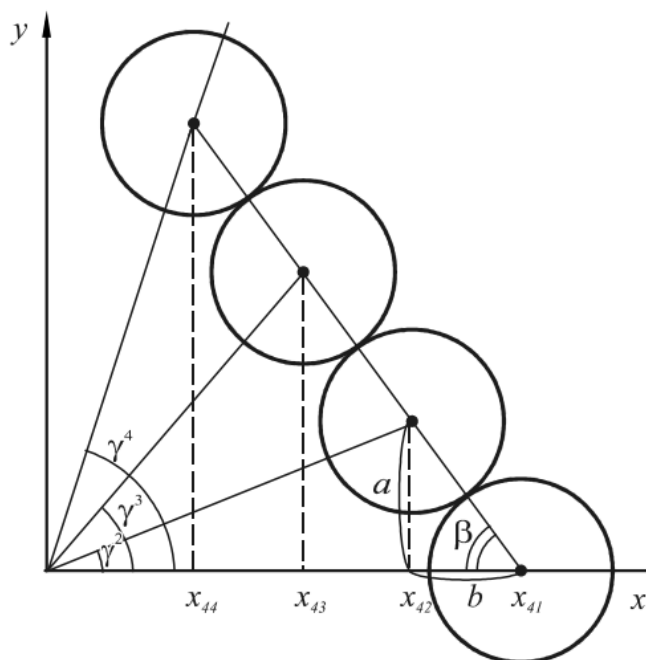


Рисунок 14 – Вычисление координат центров шаров в секторе базового пятиугольника

2.3.2 Определение координат центров атомов в секторе деформированного пятиугольника

Для того чтобы было возможно построить икосаэдрическую структуру, необходимо деформировать структуру базового пятиугольника за счет переноса части шаров с границы сектора в его вершину, что приведет к формированию неправильных десятиугольников.

Для вершины, расположенной на оси ox , когда k – нечетно, вершины с номерами $k-1, \dots, 1$ расположим под центром k -го шара так, чтобы они касались друг друга только в одной точке. То же самое сделаем с шарами

на втором луче пятиугольника, проходящем под углом α по отношению к первому (рисунок 15).

Пусть в секторе содержится m шаров. Центр первого шара сектора после перемещения опустится ниже оси абсцисс. Следовательно, всю эту

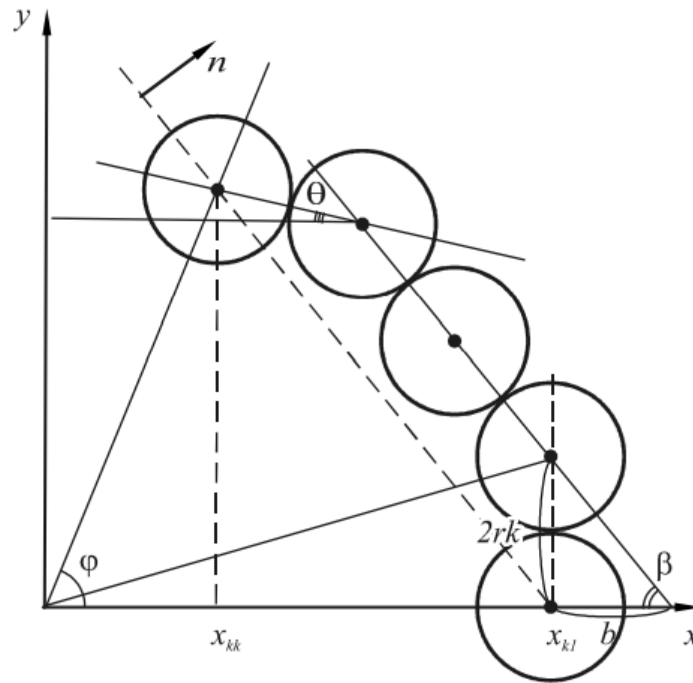


Рисунок 15 – Вычисление координат центров шаров в секторе деформированного пятиугольника

конфигурацию необходимо «поднимать» за счет ее перемещения по направлению вектора n на величину z :

$$\tilde{X}_{mj} = X_{mj} + zn, \quad (16)$$

где $n = (\cos(\pi/5); \sin(\pi/5))$, $j = k, \dots, m - k$.

Тогда, для $j = 1, \dots, k$ центры шаров вычисляются по формулам

$$\begin{cases} \tilde{x}_{mj} = \tilde{x}_{m,k+1}, \\ \tilde{y}_{mj} = \tilde{y}_{m,k+1} - 2r(k - j + 1). \end{cases} \quad (17)$$

Координаты центров вершинных шаров второго луча для $j = m - k, \dots, m$ определяются по формулам

$$\begin{cases} \tilde{x}_{mj} = \tilde{x}_{m,m-k} - 2r(j - m + k)\cos\theta, \\ \tilde{y}_{mj} = \tilde{y}_{m,m-k} + 2r(j - m + k)\sin\theta. \end{cases} \quad (18)$$

Положив $\tilde{x}_{m,1} = 0$ и решив уравнение можно получить координаты вершин деформированного пятиугольника с нечетным количеством шаров в его вершинах. Если использовать условие $\tilde{x}_{m,1} = r$, то получится деформированный пятиугольник с четным количеством шаров в его вершинах.

2.4 Параллельный алгоритм поиска оптимальных конформаций

Для стохастических алгоритмов, то есть для алгоритмов в вычислениях которых имеет место случайная величина, основным критерием достоверности результата является количество раз моделирования. Иными словами, для того, чтобы быть уверенным в том, что найден именно глобальный оптимум для конкретного числа атомов, нужно запускать алгоритм некоторое количество раз. Конечно, в любом случае нельзя абсолютно точно утверждать, что найденное решение является глобальным минимумом, однако, при достаточном большом значении числа экспериментов данная вероятность может быть очень близкой к единице.

В работе [27] отмечается, что для того, чтобы событие считать вполне достоверным, вероятность его появления в M независимых экспериментах должна составлять приблизительно $1 - 10^{-8}$. По достижении этого порога методы Монте-Карло могут по-своему «гарантировать» выдаваемые ими результаты.

Так как определить вероятность появления решения, соответствующего глобальному минимуму (обозначим эту вероятность за P) аналитически, учитывая сложность алгоритма, не представляется возможным, на основе полученных ранее результатов была посчитана оценка этой вероятности. Таким образом было подсчитано, что из 5000 полных итераций алгоритма 300 решений соответствуют глобальному минимуму, что дает право говорить о вероятности в 3-6% его получения на одной итерации.

Определим число необходимых независимых экспериментов для получения глобального минимума. Для этого посчитаем вероятность того,

что за M испытаний ни одно решение не окажется глобальным минимумом. Согласно простой формуле эта вероятность равняется $(1 - P)^M$. Соответственно, вероятность его появления будет обратной, т.е. будет равняться $1 - (1 - P)^M$. Как было определено выше, для того чтобы считать появление события достоверным, его вероятность должна равняться $1 - 10^{-8}$. Приравняем левую и правую части:

$$1 - (1 - P)^M = 1 - 10^{-8}.$$

В полученном уравнение выразим M :

$$M = \log_{(1-P)} 10^{-8}.$$

Подставим P и получим:

$$M = \log_{0.97} 10^{-8}$$

Получаем

$$M \approx 604$$

Таким образом, для получения достоверного результата необходимо запускать алгоритм не менее 600 раз. Выполнение такого количества итераций на одном ядре процессора займет от нескольких часов. Поэтому, в данной работе предлагается распараллеливать задачу с целью увеличения скорости подсчета, т.е. увеличения количества итераций алгоритма в час. Добиться этого можно растиражировав программу на несколько процессоров, проводя на них независимые испытания и в конце выбирая лучшего кандидата.

На рисунке 16 изображена блок-схема параллельного алгоритма.

2.5 Выводы по главе

В данной главе была сформулирована задача поиска оптимальных кластеров Морса, были рассмотрены алгоритмы структурной оптимизации и динамического расширения кластера, а также алгоритмы построения плотной упаковки шаров. Кроме того, был определен алгоритм параллельного поиска оптимальных конформаций.

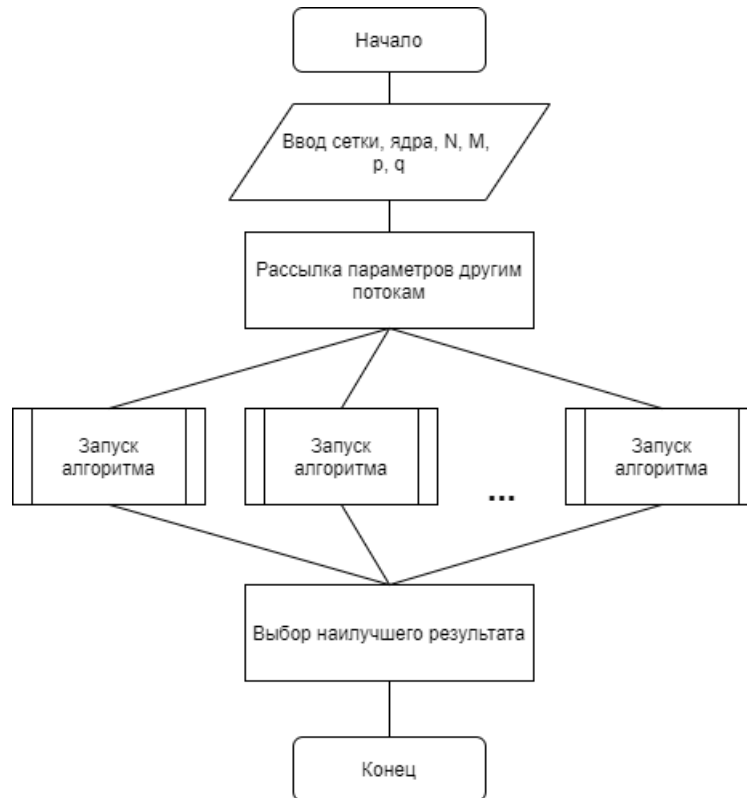


Рисунок 16 – Блок-схема параллельного алгоритма

3 Реализация разработанного алгоритма

3.1 Выбор и обоснование комплекса программных средств

В 2.5 главе было отмечено, что для получения достоверного результата необходимо большое количество испытаний и что лучшие всего для этого подходит параллельное программирование. Отсюда следует, что чем большее количество процессоров будет задействовано, тем быстрее и с большей вероятностью будет получена глобально-оптимальная конформация. Поэтому, идеальным программным средством для проведения испытаний является кластер «Сергей Королёв», расположенный в нашем университете.

Кластер «Сергей Королёв» построен на базе линейки оборудования IBM BladeCenter с использованием блейд-серверов и обеспечивает пиковую производительность 30 триллионов операций с плавающей точкой в секунду. Суммарная производительность, замеренная на отдельных частях кластера, составляет 27,235 ТФлопс. Сегодня это самый мощный суперкомпьютер в Самарской области [28].

Фактически, вычислительный кластер представляет собой совокупность компьютеров, объединенных в рамках одной сети и работающих над одной задачей. Компоненты кластера, как правило, связаны друг с другом через быстрые локальные сети. С каждым узлом работает свой экземпляр операционной системы [29].

3.2 Выбор и обоснование операционной системы

Так как программа ориентирована на работу на кластере «Сергей Королёв», в качестве операционной системы был выбран Linux. Кластер «Сергей Королёв» находится под управлением 32-битной версии Linux – Red Hat Enterprise Linux 5 на головном узле и Red Hat Enterprise Linux 6 на вычислительных узлах. Причиной, по которой на кластере используется именно эта операционная система является то, что Linux является

безоговорочным лидером, когда вопрос касается высокопроизводительных вычислений. Другая причина – это поддержка симметричных многопроцессорных архитектур. Такие системы позволяют любому процессу работать на любой задаче независимо от того, где в памяти хранятся данные для этой задачи, эффективно распределяя нагрузку [30].

Тем не менее был предусмотрен запуск программы на компьютере под управлением операционной системы Windows с возможностью распараллелить вычисления на имеющиеся ядра процессора.

3.3 Выбор и обоснование языка программирования

Не для кого не секрет, что язык C/C++ на данный момент является лидирующим в вопросе высокопроизводительных вычислений. Несмотря на то, что язык был разработан в 1969-1973 годах, он и по сей день является стандартом для реализации вычислительных алгоритмов.

Другой причиной, по которой C/C++ был выбран в качестве главного языка является то, что он дает возможность писать кроссплатформенный код, т.е. программы, написанные на нем, можно запустить из любой операционной системы, имеющей подходящий компилятор (MSVC (Microsoft Visual C) для операционной системы Windows и GCC (GNU Compiler Collection) для Linux). Если выбирать между MSVC компилятором и GCC, то обычно предпочтение отдается последнему.

3.4 Выбор и обоснование использования библиотеки PCG

Как было упомянуто в главе 1.1.8, выбор надежного ГПСЧ является необходимым в стохастических алгоритмах. Языки C и C++ имеют свои собственные встроенные генераторы случайных чисел. Они реализованы в двух отдельных функциях, которые находятся в заголовочном файле `cstdlib` [31]. Проблема по которой его не рекомендуется использовать в стохастических алгоритмах, в том числе в методе Монте-Карло, – это его сравнительно небольшой период и плохие качественные показатели.

В данной работе предлагается использование библиотеки PCG вместо библиотеки по-умолчанию, так как она обладает всеми заявленными требованиями: возможностью генерировать независимые псевдослучайные величины на разных потоках (до 2^{63} потоков), иметь легко настраиваемый период и обладать хорошими статистическими показателями.

На данный момент имеются реализации этой библиотеки под языки C/C++ и Haskell. Также планируется реализация для языка Java.

3.5 Выбор и обоснование средства распараллеливания вычислений

Выше, в главе 3.1 было предложено использовать вычислительные возможности кластера «Сергей Королёв». Благодаря увеличению числа параллельно работающих процессоров можно добиться большей скорости работы, а значит и возможности проведения большего числа испытаний.

Кластер «Сергей Королёв» относится к вычислительным системам с распределенной памятью (каждый процессор системы обладает собственной локальной памятью). Иными словами, процессоры работают независимо друг от друга. Для организации параллельных вычислений на кластере необходимо иметь возможность распределять вычислительную нагрузку между частями системы и организовывать информационное взаимодействие. Решение всех выше перечисленных вопросов обеспечивает интерфейс передачи данных MPI (message passing interface) [32].

MPI является наиболее распространённым стандартом интерфейса обмена данными в параллельном программировании: существуют его реализации для большого числа компьютерных платформ. Он используется при разработке программ для кластеров и суперкомпьютеров. Основным средством коммуникации между процессами в MPI является передача сообщений друг другу [32].

Выбор интерфейса MPI обусловлен в первую очередь тем, что он ориентирован на системы с распределенной памятью, в то время как OpenMP – на системы с общей памятью.

Под параллельной программой в рамках MPI понимается множество одновременно выполняемых процессов. Процессы могут выполняться на разных процессорах, но на одном процессоре могут располагаться и несколько процессов (в этом случае их исполнение осуществляется в режиме разделения времени). В предельном случае для выполнения параллельной программы может использоваться один процессор - как правило, такой способ применяется для начальной проверки правильности параллельной программы [32].

Каждый процесс параллельной программы порождается на основе копии одного и того же программного кода (модель SPMP). Данный программный код, представленный в виде исполняемой программы, должен быть доступен в момент запуска параллельной программы на всех используемых процессорах. Исходный программный код для исполняемой программы разрабатывается на алгоритмических языках C или Fortran с использованием той или иной реализации библиотеки MPI [32].

3.6 Выбор и обоснование среды разработки

Для написания программы под операционную систему Windows лучшим решением является использование Microsoft Visual Studio и компилятора MSVC для сборки. Средствами Visual Studio можно делать отладку программы, а также выполнять профилирование кода с целью выявления проблемных участков. В качестве среды разработки была выбрана версия 2017-го года.

Для написания программы под Linux можно воспользоваться любым текстовым редактором и компилятором GCC. Тестирование программы также следует проводить непосредственно из самой операционной системы Linux.

3.7 Дополнительное программное обеспечение

Кроме задачи поиска глобально-оптимальных конформаций кластеров Морса, немаловажным является анализ получаемых результатов. В связи с этим были написано несколько дополнительных программ, одной из которых является послойное препарирование кластера, т.е. конвертации координат центров атомов в изображение, позволяющее наглядно оценить полученный кластер. Кроме того, такое программное обеспечение просто необходимо на этапе отладки и тестирования алгоритма, потому что с его помощью можно удостовериться в правильности получаемых результатов. На изображении 17 можно увидеть послойное препарирование кластера с 275 вершинами.

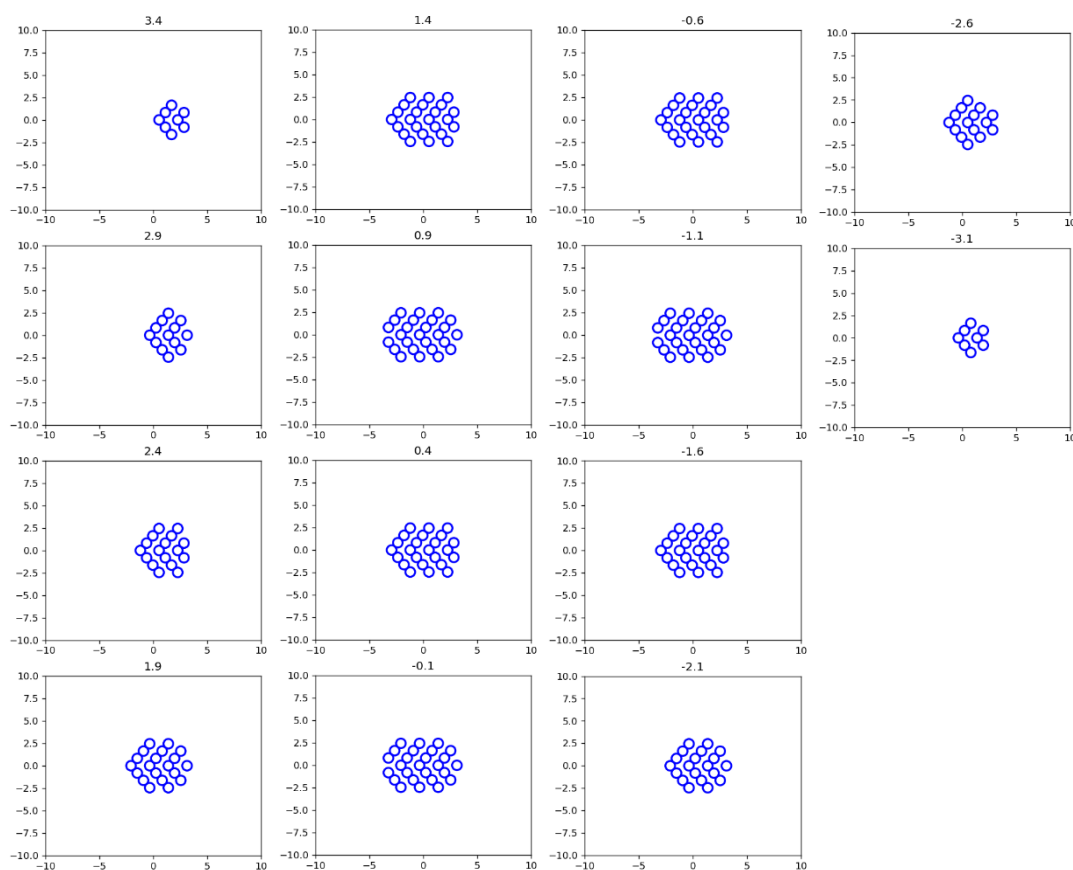


Рисунок 17 – Послойное препарирование кластера Морса с 275 вершинами

Среди других вспомогательных программ можно отметить программу для генерирования икосаэдрической сетки, описанной в главе 2.4, и программу для обработки результатов, получаемых с кластера «Сергей Королёв».

Все эти добавочные программы были написаны на языке программирования Python (версия 3.8.2) с использованием PyCharm (версия 2018.3.2) в качестве среды разработки. Выбор именно этого языка обуславливается его простотой и быстротой в написании, а также большим количеством полезных библиотек.

3.8 Выводы по главе

В данной главе были выбраны и обоснованы программные средства для работы с кластерами Морса. В качестве языка программирования были выбраны C/C++ и Python, в качестве среды разработки – Visual Studio и PyCharm, интерфейс MPI для распараллеливания вычислений, Windows и Linux в качестве операционной системы для запуска на компьютере и кластере «Сергей Королев» соответственно.

4 Вычислительные эксперименты

В рамках дипломной работы, при помощи разработанного алгоритма, был произведен поиск оптимальных кластеров Морса размерами от 268 атомов. В ходе экспериментов использовались два типа решеток: смещенная икосаэдрическая (т.н. «косоугольная») (рисунок 18(а)) и центральная икосаэдрическая (рисунок 18(б)). Обе они являются результатом «среза», продемонстрированного в главе 2.1 на рисунке 11.

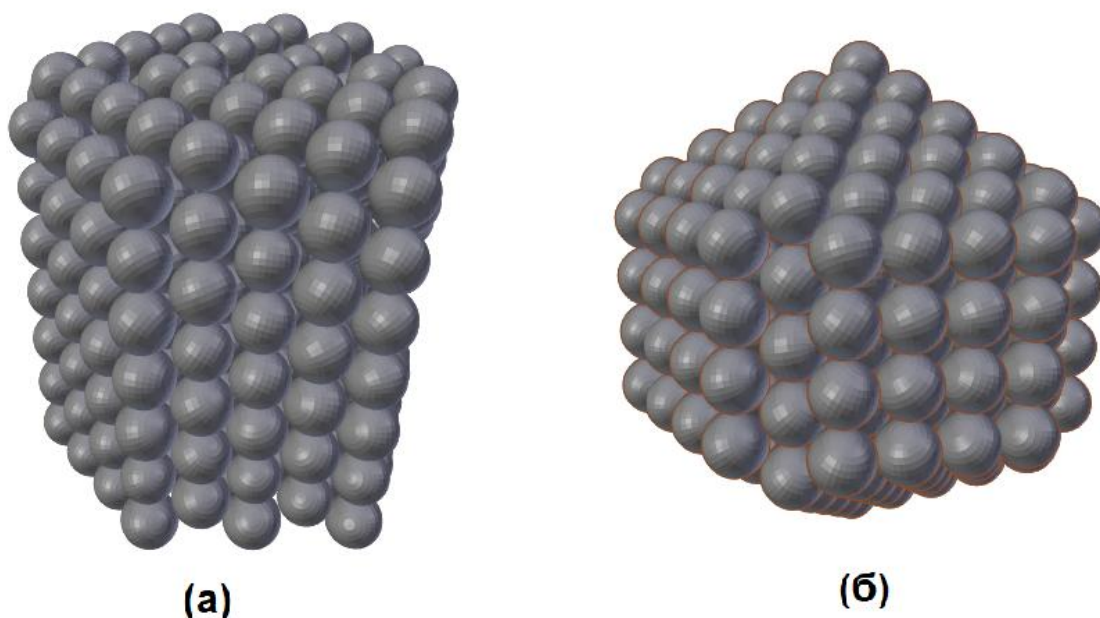


Рисунок 18 – «Косоугольная» решетка (а) и центральная решетка (б)

Согласно результатам вычислительных экспериментов была составлена таблица 2, содержащая значения обнаруженных минимальных потенциальных энергий кластеров.

Исходя из полученных результатов видно, что «косоугольная» решетка, которая являлась основой для оптимальных кластеров меньших размеров (до 266 атомов) с увеличением количества атомов все больше теряет свою эффективность. Ранее было отмечено, что для кластеров Морса не свойственно структурное однообразие и что в силу своих геометрических свойств, на разных диапазонах размеров кластера доминируют разные виды решеток. Если обратить внимание на рисунок 7, то можно увидеть, что такой

Таблица 2 – Потенциальная энергия кластеров с 268-290 атомами

Количество атомов	Потенциальная энергия взаимодействия	
	Смещенная икосаэдрическая решетка («косоугольная»)	Центральная икосаэдрическая решетка
268	-1294.449	-1295.158
269	-1299.462	-1300.131
270	-1304.475	-1306.103
271	-1310.463	-1311.117
272	-1314.488	-1317.109
273	-1319.501	-1322.082
274	-1324.513	-1327.078
275	-1329.526	-1333.067
276	-1333.539	-1339.059
277	-1338.539	-1343.066
278	-1343.552	-1348.122
279	-1347.565	-1353.095
280	-1352.565	-1358.091
281	-1357.578	-1364.079
282	-1361.591	-1370.072
283	-1365.597	-1375.028
284	-1369.604	-1379.086
285	-1372.635	-1384.108
286	-1377.654	-1390.08
287	-1382.673	-1395.092
288	-1387.691	-1401.086
289	-1392.71	-1406.059
290	-1398.729	-1411.058

переход происходит при увеличении размера кластера с 200 до 201 атома, однако, на этот раз структура меняется с центральной на «косоугольную».

На рисунке 19 можно увидеть, подсчитанную в рамках другой выпускной квалификационной работы, оптимальную потенциальную энергию кластеров меньших размеров.

Количество атомов в кластере <i>N</i>	Энергия взаимодействия атомов χ	
	Плотно упакованная структура	Икосаэдрическая структура
239	-1144,0616	-1144,5482
241	-1155,08652	-1154,113
242	-1160,10568	-1160,57356
243	-1166,11859	-1165,114
244	-1172,1300	-1171,11
245	-1176,1500	-1175,113
246	-1181,14472	-1179,96957
247	-1186,1930	-1184,3192
248	-1192,1703	-1190,7000
249	-1197,1831	-1195,1001
250	-1203,19575	-1199,999
251	-1207,2207	-1205,9481
252	-1212,2456	-1211,11133
253	-1217,26437	-1216,14
254	-1223,23465	-1222,126
255	-1228,2897	-1227,14
...	...	...
260	-1254,299	-
266	-1284,4123	-1283,9000
...	...	...
318	-	-1560,11539
...	...	...
389	-	-1935,87887

Рисунок 19 – Оптимальная энергия кластеров меньших размеров

На графике, изображенном на рисунке 20, можно отследить закономерное увеличение разрыва между энергиями разных структур. Для того чтобы объяснить такое значительное ухудшение косоугольной структуры обратимся к трехмерному изображению (рисунок 21). Невооруженным глазом можно заметить, что кластер (а), кластер с центральной икосаэдрической структурой, имеет более округлую форму, чем кластер (б), который был получен косоугольной решеткой. Это ухудшение можно объяснить тем, что добавление нового атома на новый слой оказывается менее выгодным.

По большей части, при формировании каждого последующего кластера прослеживается наследуемая группа атомов, и некоторые «отзеркаленные» ее участки (рисунок 22).

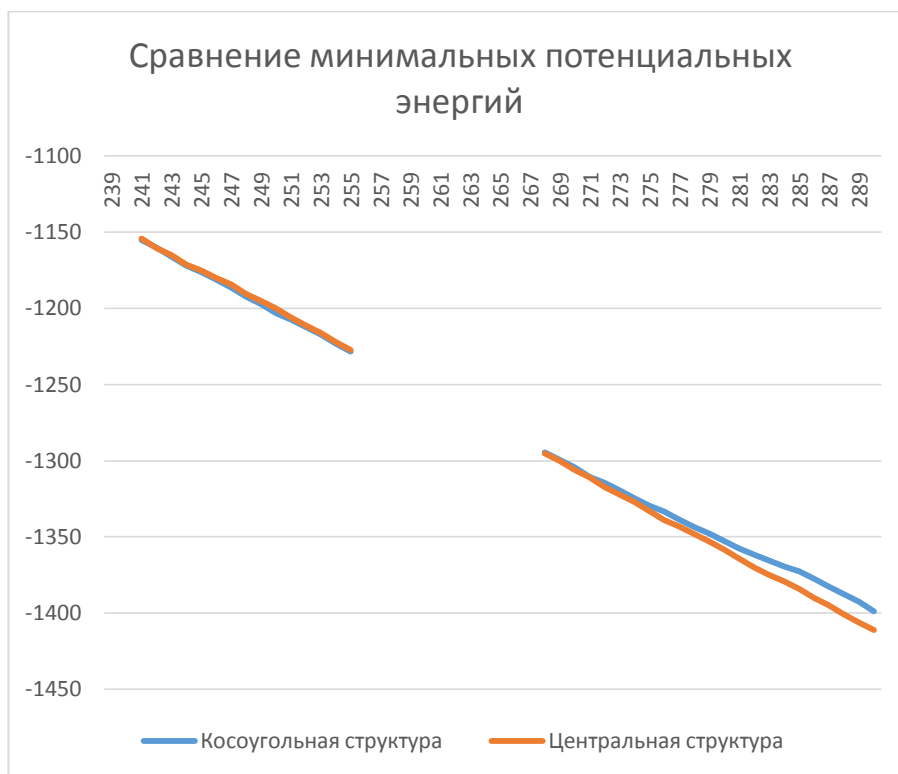


Рисунок 20 – Сравнение потенциальной энергии двух типов структур

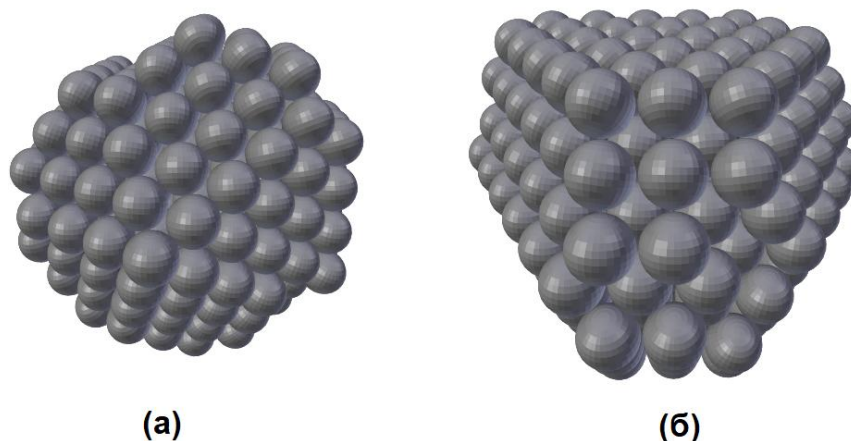


Рисунок 21 – Сравнение двух кластеров размером 274

Если посмотреть на структуру полученных кластеров, можно заметить их симметричность относительно плоскости xOz (рисунок 23). Наблюдение подтверждается и на кластерах меньших размеров. Это позволяет выдвинуть предположение об еще одном возможном упрощении задачи – симметричном динамическом расширении кластера. При таком упрощении, атомы

добавляются не по одному, а по два в симметричные относительно плоскости позиции, что позволило бы смотреть на кластер, как если бы в нем было в два раза меньше атомов.

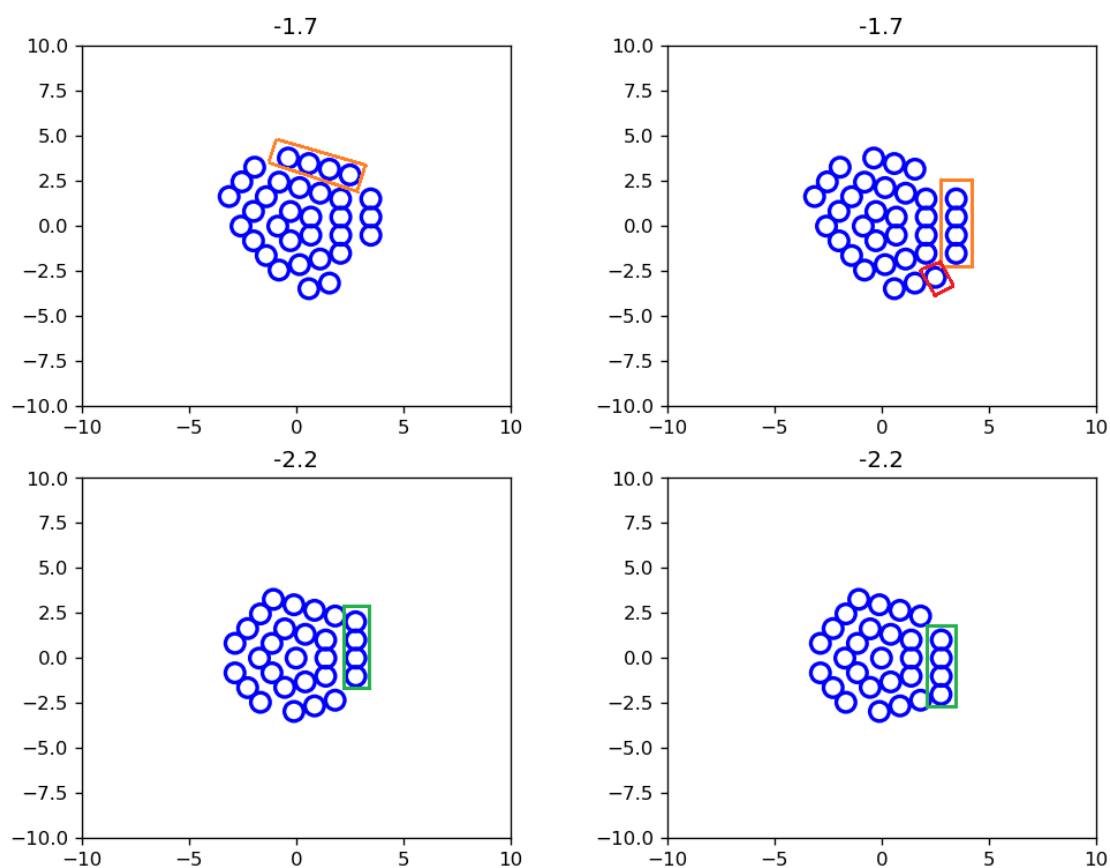


Рисунок 22 – Изменения в кластерах 289 и 290 атомов

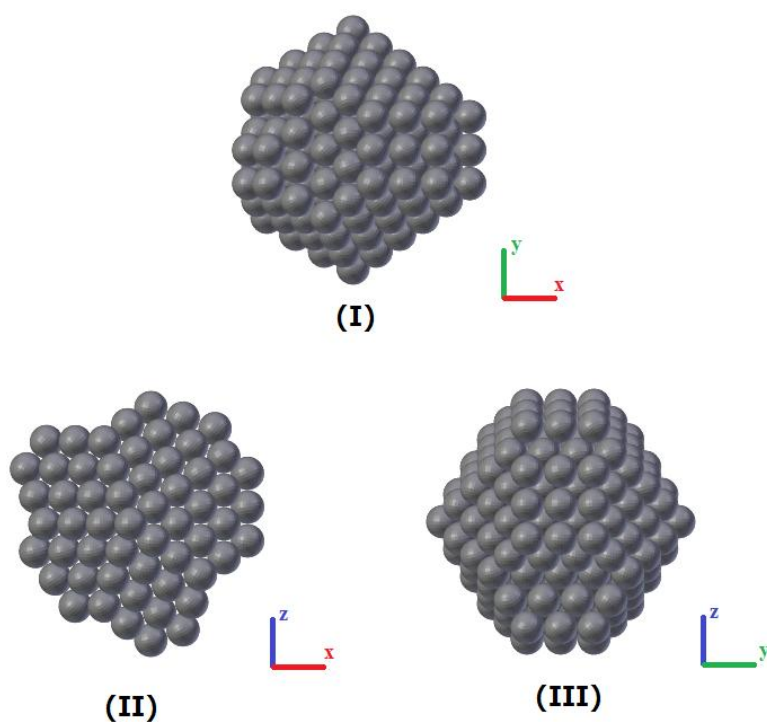


Рисунок 23 – 269-атомный кластер. (I) – вид сверху, (II) – вид спереди, (III) – вид справа

Для того, чтобы сократить количество вычислений, в программе есть возможность выделить группу атомов, которая будет неизменной для каждой итерации, т.н. ядро кластера. В связи с этим, от того, как будет выбрано ядро, зависит получаемый результат. Есть два варианта: центральным слоем ядра будет четный слой или нечетный (см. рисунок 13). От того каким будет этот слой зависит возможный энергетический минимум.

В ходе экспериментов над кластером, состоящим из 251 атома, было установлено, что наиболее предпочтительным является нечетный слой, а минимумы энергии при четном и нечетном слое составили -1201.123 и -1207.075 соответственно.

Таким образом, в ходе экспериментов были получены адекватные результаты, что дает право полагать, что разработанный алгоритм работает правильно. Результаты стыкуются с полученными ранее и опубликованными в [9]. Также можно заметить линейный рост потенциальной энергии, что тоже свидетельствует о правильной работе алгоритма. Код программы приведён в приложении А.

4.1 Сравнение последовательной и параллельной версии

Как уже было сказано выше, больше число экспериментов означает большую достоверность результатов и что добиться этого можно распараллелив программу на несколько процессоров.

Параллельная программа устроена таким образом, что отдельные модули производят независимые итерации и сохраняют результат в общую память. По окончании выполнения (когда закончится таймер или программа будет принудительно остановлена), среди всех найденных конформаций отбирается наилучшая и сохраняется в файл. Иными словами, так как ничто не мешает процессам работать отдельно друг от друга, т.е. не затрачивается

время на синхронизацию и простаивания одного из потоков, следует ожидать линейный прирост производительности, зависящий от числа вычислительных ядер.

Чтобы проверить эту гипотезу взглянем на результаты, полученные с использованием одного, двух и трех ядер процессора компьютера. Результаты приведены в таблице 3.

Таблица 3 – Сравнение работы программы на разном числе ядер компьютера

Количество вычислительных ядер	Количество полных итераций	Общее время, мин	Скорость, ит/мин	Ускорение	Ускорение на одном ядре
1	42	46	0.913	1	1
2	68	46	1.472	1.61	0.8
3	92	46	2	2.19	0.73

Также, в таблице 4 приведены результаты, полученные на кластере «Сергей Королёв» при запуске программы на одном восьми и шестнадцати потоках.

Таблица 4 – Сравнение работы программы на разном числе ядер кластера

Количество вычислительных ядер	Количество полных итераций	Общее время, мин	Скорость, ит/мин	Ускорение	Ускорение на одном ядре
1	58	70	0.829	1	1
8	366	70	5.228	6.3	0.78
16	683	70	9.757	11.9	0.74

Стоит учитывать, что вычислительный кластер состоит из разных процессоров. На рисунках 24 и 25 можно увидеть, как меняется количество полных итераций в зависимости от номера процессора.

Исходя из полученных результатов, можно сделать вывод, что распараллеленная программа теряет приблизительно 25% скорости на каждом вычислительном ядре. Иными словами, используя 10 процессов мы получим ускорение не в 10 раз, а в 7.5. Учитывая многогранность причин, влияющих на вычислительный эксперимент (разнородность процессоров,

сторонние процессы в случае запуска не компьютере и т.д.), более точное измерение произвести не представляется возможным.

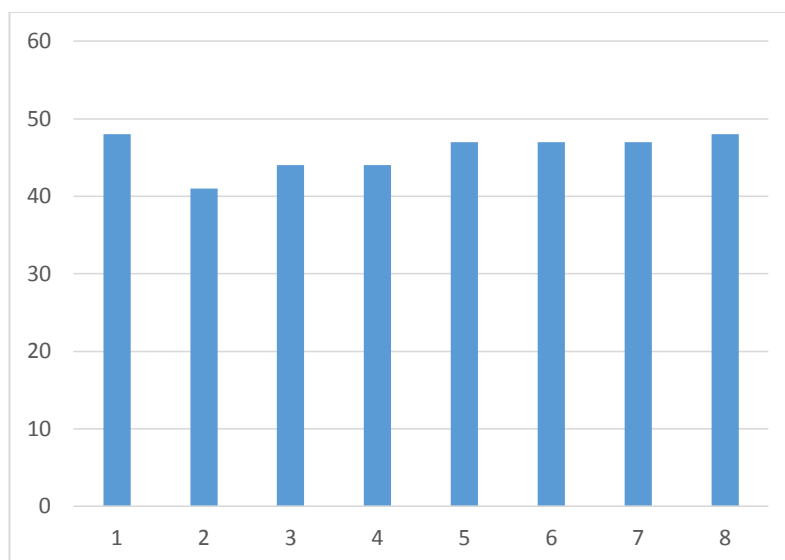


Рисунок 24 – Количество итераций каждого процессора (8 ядер)

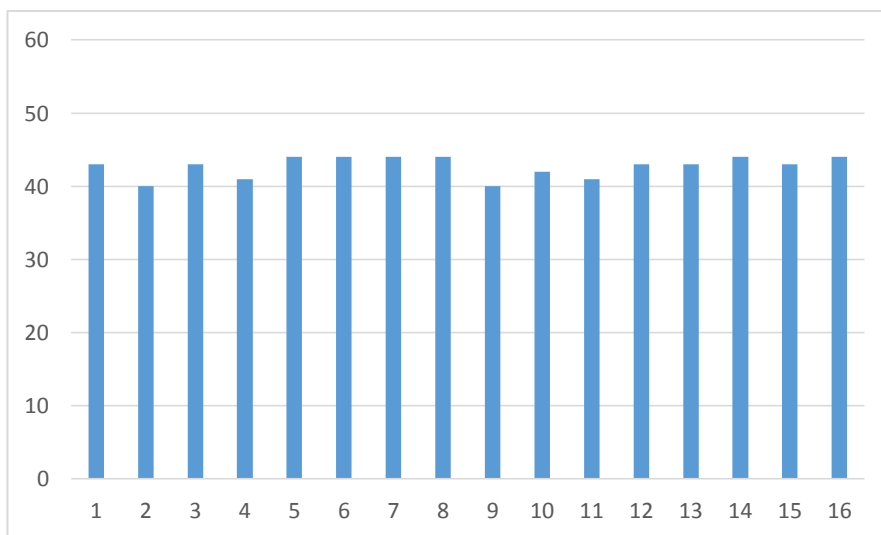


Рисунок 25 – Количество итераций каждого процессора (16 ядер)

4.2 Временная сложность алгоритма

Как было отмечено выше, фактически, алгоритм имеет линейный рост времени, затрачиваемого на построение кластера до нужных размеров, однако, на практике, зависимость не совсем линейная. Дополнительные временные затраты вносит то, что при растущем числе атомов кластера растет и количество связей между атомами. Иными словами, увеличивается число возможных позиций для добавления нового атома.

В таблице 5 приведены временные расчеты для кластеров, содержащих от 80 до 300 атомов.

		Количество атомов кластера					
		80	100	120	140	160	180
t, с	1	4.256	5.5786	8.6656	14.7496	17.7438	21.7656

Таблица 5 – Среднее время на одну итерацию алгоритма для различных кластеров.

Таблица 5 – Продолжение

	Количество атомов кластера					
	200	220	240	260	280	300
1	26.332	31.362	39.4892	46.4838	53.2644	63.4

График зависимости времени от числа атомов кластера приведен на рисунке 26.

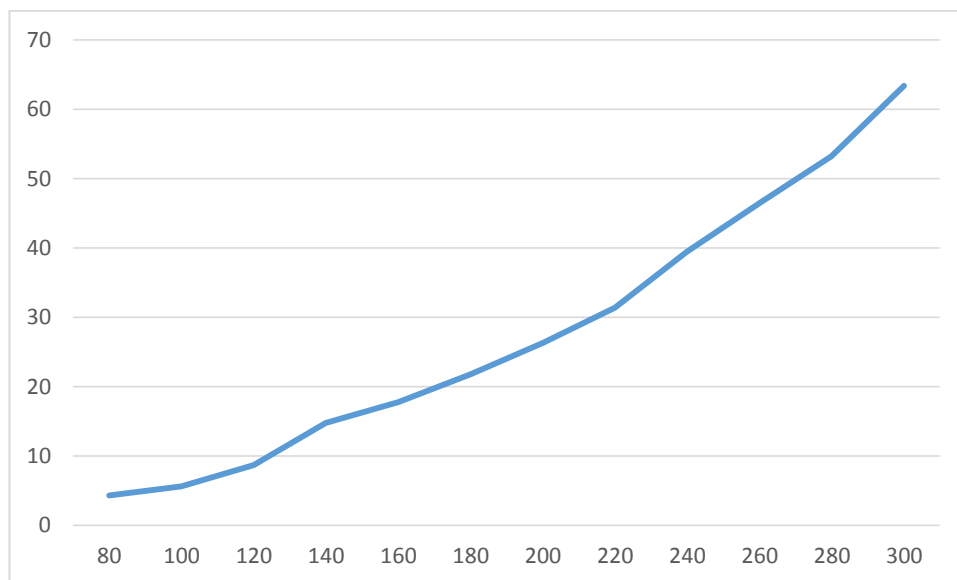


Рисунок 26 – Зависимость времени одной итерации от количества атомов кластера

Наибольший коэффициент корреляции имеет квадратичная регрессия, соответствующая уравнению $y = 0.0008x^2 - 0.0243x + 1.3045$. Как можно заметить, коэффициент квадратичного роста достаточно мал, что говорит о медленном росте.

Однако, не стоит забывать, что количество испытаний должно быть большим, и, что даже для 280-атомного кластера время, которое потратится на 600 итераций составит приблизительно 9 часов.

Так как задача имеет рост сложности, близкий к линейному, применение параллельных вычислений имеет практическое значение. Алгоритм можно использовать и для кластеров состоящих из большего числа атомов, и время при этом не будет запредельно высоким.

4.3 Выводы по главе

В данной главе рассматривались эксперименты над кластерами Морса. Проведен сравнительный анализ последовательной и параллельной версий. Выполнена оценка временной сложности алгоритма. Также было выдвинуто предположение о возможной модификации алгоритма.

Полученные результаты подтверждают адекватность используемого метода.

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной работы был написан параллельный стохастический алгоритм для поиска оптимальных конформаций кластеров Морса.

В первом разделе были приведены основные понятия и определения предметной области. Рассмотрен атомный кластер с точки зрения объекта моделирования, некоторые понятия, связанные с ним, а также модели межатомного взаимодействия. Рассмотрены уже существующие методы поиска глобально-оптимальных атомных конформаций. Рассмотрена идея геометрически обоснованного метода.

Во втором разделе приведен алгоритм динамического расширения и разработана его параллельная версия. Также приведены правила формирования икосаэдрической упаковки шаров, на которой ищется глобальной-оптимальная конформация.

В третьей части приведены обоснование выбора каждого программного продукта, необходимого для реализации разработанного алгоритма.

В четвертой части приведены результаты экспериментов, проведенных над кластерами с использованием разработанного параллельного стохастического алгоритма формирования начальных конформаций. Проведен сравнительный анализ последовательной и параллельной версий и выполнен анализ временной сложности.

Разработанный алгоритм позволяет осуществлять поиск глобально-оптимальных, или близких к глобально-оптимальным, конформаций для кластеров Морса больших размеров. Вычислительные эксперименты подтверждают рациональность используемого метода и его возможность применения к кластерам еще больших размеров.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Фолдинг белка [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Фолдинг_белка (дата обращения: 10.01.2020).
- 2 Folding@home [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: <https://ru.wikipedia.org/wiki/Folding@home> (дата обращения: 10.01.2020).
- 3 Коварцев А.Н. К вопросу об эффективности параллельных алгоритмов глобальной оптимизации функций многих переменных / Коварцев А.Н., Попова-Коварцева Д.А. – Компьютерная оптика. 2011. №2. Т. 35. С. 256-261.
- 4 Коварцев А.Н. Эволюционный детерминированный алгоритм глобальной оптимизации атомных кластеров Морса // Компьютерная оптика. 2015. №2. Т. 39. С. 234-240.
- 5 Смирнов Б.М. Кластеры и фазовые переходы // Успех физических наук. 2007. Т.177. №4. С. 369-373.
- 6 Кластер (химия) [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: [https://ru.wikipedia.org/wiki/Кластер_\(химия\)](https://ru.wikipedia.org/wiki/Кластер_(химия)) (дата обращения: 14.02.2020).
- 7 Pereira F. B. Analysis of Crossover Operators for Cluster Geometry Optimization [Электронный ресурс] / Pereira F. B., Marques J. M. C. – Chapter, November 2010. URL: <https://www.researchgate.net/publication/226395707> (дата обращения: 16.02.2020).
- 8 Коварцев А.Н. Геометрически обоснованный метод формирования атомных кластеров Морса // Компьютерная оптика. 2017. Т. 41. №1 С. 118-125.
- 9 Morse Cluster: Table of global minima (7-240) [Электронный ресурс] // The Cambridge Cluster Database: [сайт]. URL: <http://staff.ustc.edu.cn/~clj/morse/table.html> (дата обращения: 25.02.2020).

10 Doye J.P.K. The effect of the range of the potential on the structures of clusters / Doye J.P.K., Wales D.J., Berry R.S. – Journal of Chemical Physics. 1995. V.103. P.4234-4289.

11 Morse potential [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://en.wikipedia.org/wiki/Morse_potential (дата обращения: 02.03.2020).

12 Орлянская И.В. Современные подходы к построению методов глобальной оптимизации // Электронный журнал «Исследовано в России». 2002. С. 2097-2108.

13 Иванов А.С. Основные принципы молекулярного конформационного анализа для медико-биологов // Электронный журнал «Биомедицинская химия». 2007, Т.53. вып.6. с.213-728.

14 Monte Carlo method [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://en.wikipedia.org/wiki/Monte_Carlo_method (дата обращения: 02.03.2020).

15 Эффективность по Парето [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Эффективность_по_Парето (дата обращения: 02.03.2020).

16 Линейный конгруэнтный метод [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Линейный_конгруэнтный_метод (дата обращения: 05.03.2020).

17 Permuted congruential generator [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://en.wikipedia.org/wiki/Permuted_congruential_generator (дата обращения: 05.03.2020).

18 PCG, A Family of Better Random Number Generators [Электронный ресурс] // URL: <https://www.pcg-random.org/index.html> (дата обращения: 05.03.2020).

19 Генетический алгоритм [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Генетический_алгоритм (дата обращения: 08.03.2020).

20 Storn R. Differential Evolution. A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces / Storn R., Price K. – Journal of Global Optimization. 1997. V. 11. P. 341-359.

21 Moloi N.P. An Iterative Global Optimization Algorithm for Potential Energy Minimization / Moloi N.P., Ali M.M. – Journal of Computational Optimization and Applications. 2005. V. 30. P. 1-14.

22 Basin-hopping [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: <https://en.wikipedia.org/wiki/Basin-hopping> (дата обращения: 15.03.2020).

23 Cheng L. Funnel hopping: searching the cluster potential energy surface over the funnels / Cheng L, Feng Y, Yang J, Yang J. – J Chem Phys. 2009. №130(21).

24 Floudas C.A. Global optimization in the 21st century: Advances and challenges / Floudas C.A., Akrotirianakis I.G., Caratzoulas S., Meyer C.A., Kallrath J. – Computers and Chemical Engineering. 2005. V.29. P. 1185-1202.

25 Коварцев А.Н. Правила формирования начальных приближений конформаций атомных кластеров Морса на основе геометрически обоснованного метода / Коварцев А.Н., Климашова Г.П. – Международная конференция и молодёжная школа «Информационные технологии и нанотехнологии» (ИТНТ-2018). 2018. С. 1635-1644.

26 Семенов К.К. Достоверность результатов применения метода Монте-Карло в задачах интервального анализа // Вычислительные технологии. Т. 21. № 2. 2016. С. 42-52.

27 Суперкомпьютер «Сергей Королёв» [Электронный ресурс] // Суперкомпьютерный центр Самарского университета: [сайт]. URL: <http://hpc.ssau.ru/node/6> (дата обращения: 10.04.2020).

28 Компьютерный кластер [Электронный ресурс] URL: https://ru.qwe.wiki/wiki/Computer_cluster (дата обращения: 10.04.2020).

29 Симметричная многопроцессорность [Электронный ресурс] // Википедия: всемирная энциклопедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Симметричная_многопроцессорность (дата обращения: 10.04.2020).

30 Генерация случайных чисел [Электронный ресурс] // Уроки C++: [сайт]. URL: <https://ravesli.com/urok-71-generatsiya-sluchajnyh-chisel-funktsii-srand-i-rand/> (дата обращения: 14.04.2020).

31 Параллельное программирование на основе MPI [Электронный ресурс] // ИНТУИТ. Национальный открытый университет: [сайт]. URL: <https://www.intuit.ru/studies/courses/542/398/lecture/9177?page=1> (дата обращения: 14.04.2020).

ПРИЛОЖЕНИЕ А

Код программы

```
#include <math.h>
#include "Stype.h"
#include <mpi.h>

int FmorsM(XSIM *,int *, int *,double *);
int Bsort(MASSIV *,MASSIV *,MASIND *,MASIND *,int *);

int KRIT_MS(XSIM *Xsimp,XSIM *XS, MASIND *Top0,double *FRb,TOPDE *TopMax,double
*Dop)
{
int r;
MPI_Comm_rank(MPI_COMM_WORLD, &r);
TOPDE TopM;
MASIND A,IndF;
MASSIV FT,FTS;
int NT0,NTM,I,J,Tbaz,NX,NX2,Tmin,F,Im;
double FRI2,FR,Fmin;
// Top0 - свободные вершины
// TopMax - вершины смежные первого уровня
F=1;
NT0=(*Top0)[0];
NTM=(*TopMax)[0][0];
NX=(*XS)[0][0]; NX2=NX+2;

#pragma omp parallel for schedule(static)
for(I=1;I<=NTM;I++) //Головной цикл
{
Tbaz=(*TopMax)[0][I];
//Просмотр без Tbaz
Fmin=1e77;
for(J=1;J<=(*Top0)[0];J++)
{
if((*Top0)[J]!=Tbaz)
{
(*XS)[NX+1][0]=(*Xsimp)[Tbaz][0];
(*XS)[NX+1][1]=(*Xsimp)[Tbaz][1];
(*XS)[NX+1][2]=(*Xsimp)[Tbaz][2];
(*XS)[NX+2][0]=(*Xsimp)[(*Top0)[J]][0];
(*XS)[NX+2][1]=(*Xsimp)[(*Top0)[J]][1];
(*XS)[NX+2][2]=(*Xsimp)[(*Top0)[J]][2];
}
FmorsM(XS,&NX,&NX2,&FRI2);
FR=(*FRb)+FRI2;
if(Fmin>FR)
{
Fmin=FR;Tmin=J;
//printf("%d: %e\n",r, Fmin);
}
}
(*TopMax)[1][I]=(*TopMax)[2][I];
(*TopMax)[2][I]=Fmin;
}
```

```

for(I=1;I<=NTM;I++)
    FT[I]=(*TopMax)[2][I];

FT[0]=FTS[0]=NTM;
Bsort(&FT,&FTS,&A,&IndF,&F);
for (I = 1; I <= NTM; I++)
{
    TopM[0][I] = (*TopMax)[0][IndF[I]];
    TopM[1][I] = (*TopMax)[1][IndF[I]];
    TopM[2][I] = (*TopMax)[2][IndF[I]];
}
for(I=1;I<=NTM;I++)
{
    if(fabs(FTS[I]-FTS[1])<=(*Dop)))
    {
        (*TopMax)[0][I]= TopM[0][I];
        (*TopMax)[1][I]= TopM[2][I];
        (*TopMax)[2][I]= TopM[1][I];
        Im=I;
    }
}
(*TopMax)[0][0]=Im;
return(1);
}

```

```

#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "stype.h"
#include <mpi.h>

```

```

int Fmors(XSIM *Xs,double *FR_)
{
    double r0,FR,R,F;
    double XI[3],XJ[3];
    int I,J,Nt;
    static Iopt = 0;
    r0=r0_;
    FR=0;
    Nt=(*Xs)[0][0];
    for(J=1;J<=Nt;J++)
    {
        XJ[0]=(*Xs)[J][0];
        XJ[1]=(*Xs)[J][1];
        XJ[2]=(*Xs)[J][2];
        for(I=J+1;I<=Nt;I++)
        {
            XI[0]=(*Xs)[I][0];
            XI[1]=(*Xs)[I][1];
            XI[2]=(*Xs)[I][2];
            R=(XJ[0]-XI[0])*(XJ[0]-XI[0])+(XJ[1]-XI[1])*(XJ[1]-XI[1])+(XJ[2]-XI[2])*(XJ[2]-
            XI[2]);
            R=sqrt(R);
            F=exp(r0*(1-R))*(exp(r0*(1-R))-2);
            if(F>100000)
                r0 = r0_;
        }
    }
}

```

```

        FR=FR+F;
    }
}

(*FR_)=FR;

int r;
MPI_Comm_rank(MPI_COMM_WORLD, &r);

errno_t err;
FILE *stream;
char buf[100];
snprintf(buf, 100, "C:\\MORZC\\temp\\temp%e_%d.TXT", FR, r);
err = fopen_s(&stream, buf, "w+");
int i; for (i = 1; i <= (int)(Xs[0][0][0]); i++)
    fprintf(stream, "%e %e %e\n", (*(XSIM *) (Xs))[i][0], (*(XSIM *) (Xs))[i][1], (*(XSIM *) (Xs))[i][2]);
fclose(stream);

printf("%e\n", FR);
if (FR <= -1284.4)
{
    Iopt++; printf("Opt: %d\n", Iopt);
}
return 1;
}

#include "graph.h"
#include <stdio.h>

int GraphMV(void* P[],int jroot,int Ntop, DEFPRED *ListPred,DEFTOP *ListTop,DEFGRAF
*ListGraf)
{
    int jfirst,jlast,j,jPred,jTop,Fpred;
    int (*Tp)(void*);
    /*,л§®Ÿ €®a-ГŸ®© ŸГaиЁ-л*/
    //printf("- 3 «®-- 3 «\n");
    Tp = ListTop[jroot].Top; (*Tp)(P[jroot]);
    // printf(",ГaиЁ- : %s \n",ListTop[jroot].NameTop);

    jfirst = ListTop[jroot].FirstDef;
    jlast = ListTop[jroot].LastDef;
    BEGIN:
    for(j = jfirst; j <= jlast; j++)
    {
        jPred = ListGraf[j].NambPred;

        Tp = ListPred[jPred].Predicate; Fpred = (*Tp)(P[jPred+Ntop]);

        if( Fpred==1 )

            { jTop = ListGraf[j].NambTop;
            // printf("rrrr : %s \n",ListTop[jTop].NameTop);
              Tp = ListTop[jTop].Top; (*Tp)(P[jTop]);
            // printf(",ГaиЁ- : %s \n",ListTop[jTop].NameTop);

              jfirst = ListTop[jTop].FirstDef;

```

```

        jlast = ListTop[jTop].LastDef;

        if( (jlast)==-77)
            goto ENDGRAF;
        goto BEGIN;
    }
}
// printf("€§ ъГайЁ-л %s -Гв а §аГиГ--®J® ъле®ϣ \n",ListTop[jTop].NameTop);
ENDGRAF:
return 1;
}
#include <stdio.h>
#include <stdlib.h>
#include "stype.h"
#include "pcg/pcg_basic.h"

int Probab(MASIND *TopS,MASIND *Top,int *NatomF)
{
    pcg32_random_t rng;
    pcg32_srandom_r(&rng, time(NULL), (intptr_t)&rng);

    int i,k,j,Nk,iT,NP;
    MASIND ToS;

    //Равновероятная (баесовская модель)

    NP=(*TopS)[0];
    for(i=1;i<=NP;i++)
        ToS[i]=(*TopS)[i];

    if(NP==1)
    {
        (*Top)[0]=1;
        (*Top)[1]=0;
        (*Top)[2]=(*TopS)[2];
        return 1;
    }

    k=2;
    (*Top)[0]=(*NatomF)+1;
    (*Top)[1]=0;
    Nk=(*NatomF);

    for(i=1;i<=Nk;i++)
    {
        iT=1 + pcg32_random_r(&rng)%NP;
        (*Top)[k]=ToS[iT]; k++;

        for(j=iT;j<NP;j++)
            ToS[j]=ToS[j+1];
        NP--;
    }

    return 1;
}
#include "stype.h"

```

//Модуль 0 агр Динамическое расширение

```
int aaaaaaas(void *p[])
{
    MASIND *pt1,*pt0;
    int k,n2;

    (*(MASIND *) (p[0]))[0]=(*(MORSTR *) (p[1])).Top1[0];
    (*(int *) (p[2]))=(*(MASIND *) (p[0]))[0];

    for(k=1;k<=(*(int *) (p[2]));k++)
        (*(MASIND *) (p[0]))[k]=(*(MORSTR *) (p[1])).Top1[k];

    n2=(*(MASIND *) (p[3]))[0]=(*(MORSTR *) (p[1])).TopZ[0];

    for(k=1;k<=n2;k++)
        (*(MASIND *) (p[3]))[k]=(*(MORSTR *) (p[1])).TopZ[k];

    (*(int *) (p[4]))=(*(MASIND *) (p[5]))[0]=(*(MORSTR *) (p[1])).Top0[0];

    for(k=1;k<=(*(int *) (p[4]));k++)
        (*(MASIND *) (p[5]))[k]=(*(MORSTR *) (p[1])).Top0[k];

    pt1=p[0];pt0=p[5];
    return 1;
}
```

#include "stype.h"

```
int aaaaaaad(void *p[])
{
    MASIND *pt1,*pt0;
    int k,j;
    pt1=p[1];pt0=p[2];
    (*(int *) (p[0]))=(*(MASIND *) (p[1]))[0];
    for(k=1;k<=(*(int *) (p[0]));k++) (*(MASIND *) (p[2]))[k]=(*(MASIND *) (p[1]))[k];
    j=k;
    for(k=2;k<=(*(int *) (p[3]));k++)
        { (*(MASIND *) (p[2]))[j]=(*(MASIND *) (p[4]))[k]; j++;}
    (*(MASIND *) (p[2]))[0]=(*(int *) (p[5]))=j-1; (*(int *) (p[6]))=1;
    return 1;
}
```

#include "stype.h"

```
int aaaaaaae(void *p[])
{
    int k,j;for(k=1;k<=(*(int *) (p[0]));k++)
    { (*(XSIM *) (p[1]))[k][0]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[k]][0];
      (*(XSIM *) (p[1]))[k][1]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[k]][1];
      (*(XSIM *) (p[1]))[k][2]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[k]][2];}
    (*(XSIM *) (p[1]))[0][0]=(*(int *) (p[0]));
    (*(int *) (p[4]))=1; (*(int *) (p[5]))=(*(int *) (p[0]))+1;
    (*(MASSIV *) (p[6]))[0]=(*(int *) (p[7])); (*(MASSIV *) (p[8]))[0]=(*(int *) (p[7]));
    return 1;
}
#include "stype.h"
```



```

int aaaaaaah(void *p[])
{
    (*(XSIM *) (p[0]))[(*(int *) (p[1]))][0]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[(*(int *) (p[4]))]][0];
    (*(XSIM *) (p[0]))[(*(int *) (p[1]))][1]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[(*(int *) (p[4]))]][1];
    (*(XSIM *) (p[0]))[(*(int *) (p[1]))][2]=(*(XSIM *) (p[2]))[(*(MASIND *) (p[3]))[(*(int *) (p[4]))]][2];
    return 1;
}
#include "stype.h"

int aaaaaaas(void *p[])
{
    MASIND *p0;
    int k,i;
    p0=p[1];
    i=1;for(k=1;k<(*(int *) (p[0]));k++)
        if(*(MASIND *) (p[1]))[k]!=(*(int *) (p[2]))){(*(MASIND *) (p[1]))[k]=(*(MASIND *) (p[1]))[i];i++;}
    else
    {i++;(*(MASIND *) (p[1]))[k]=(*(MASIND *) (p[1]))[i];i++;}
    (*(MASIND *) (p[1]))[0]=(*(int *) (p[0]))-1;

    return 1;
}
#include "stype.h"
//Модуль 2 агр Формирование начального приближения
int aaaaaaba(void *p[])
{
    MASIND *ptY, *ptop1, *ptop11;
    int k,j;(*(int *) (p[0]))=(*(MASIND *) (p[1]))[0];(*(int *) (p[2]))=(*(MASIND *) (p[3]))[0];
    for(k=1;k<=(*(int *) (p[0]));k++)(*(MASIND *) (p[4]))[k]=(*(MASIND *) (p[1]))[k];
    j=k;
    for(k=2;k<=(*(int *) (p[2]));k++)
        {(*(MASIND *) (p[4]))[j]=(*(MASIND *) (p[3]))[k]; j++;}
    (*(MASIND *) (p[4]))[0]=(*(int *) (p[5]))-j-1;
    ptY = p[1]; ptop1 = p[3]; ptop11 = p[4];
    return 1;
}
#include <stdio.h>
#include "stype.h"

int aaaaaabl(void *p[])
{
    int i; for(i=1;i<=(*(int *) (p[0]));i++)
        fprintf((*(PFILE *) (p[1])), "%e %e %e\n",(*(XSIM *) (p[2]))[i][0],(*(XSIM *) (p[2]))[i][1],(*(XSIM *) (p[2]))[i][2]);
    for(i=1;i<=(*(int *) (p[0]))-1;i++)
        fprintf((*(PFILE *) (p[1])), "%d,",(*(MASIND *) (p[3]))[i]);
    fprintf((*(PFILE *) (p[1])), "%d\n",(*(MASIND *) (p[3]))[(*(int *) (p[0]))]);
    return 1;
}
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "stype.h"

int FmorsM(XSIM *Xs,int *NTY, int *NT11,double *FR_)
{
    double r0,FR,R,F;

```

```

double XI[3],XJ[3];
int I,J;

r0=r0_;
FR=0;
for(I=(*NTY)+1;I<=(*NT11);I++)
{
    XI[0]=(*Xs)[I][0];
    XI[1]=(*Xs)[I][1];
    XI[2]=(*Xs)[I][2];
    for(J=1;J<=(*NTY)-1;J++)
    {
        XJ[0]=(*Xs)[J][0];
        XJ[1]=(*Xs)[J][1];
        XJ[2]=(*Xs)[J][2];
        R=(XJ[0]-XI[0])*(XJ[0]-XI[0])+(XJ[1]-XI[1])*(XJ[1]-XI[1])+(XJ[2]-XI[2])*(XJ[2]-
XI[2]);
        R=sqrt(R);
        F=exp(r0*(1-R))*(exp(r0*(1-R))-2);
        FR=FR+F;
    }
}
for(I=(*NTY);I<=(*NT11);I++)
{
    XI[0]=(*Xs)[I][0];
    XI[1]=(*Xs)[I][1];
    XI[2]=(*Xs)[I][2];
    for(J=I+1;J<=(*NT11);J++)
    {
        XJ[0]=(*Xs)[J][0];
        XJ[1]=(*Xs)[J][1];
        XJ[2]=(*Xs)[J][2];
        R=(XJ[0]-XI[0])*(XJ[0]-XI[0])+(XJ[1]-XI[1])*(XJ[1]-XI[1])+(XJ[2]-XI[2])*(XJ[2]-XI[2]);
        R=sqrt(R);
        F=exp(r0*(1-R))*(exp(r0*(1-R))-2);
        FR=FR+F;
    }
}
(*FR_)=FR;
return 1;
}#include "stype.h"

int BTOP(FSIM *Fsimp,MASIND *Top0,MASIND *Top1)
{
    int i,j,k,NT0;
    MASIND T0;

    NT0=(*Fsimp)[0];
    k=1;
    for(i=1;i<=NT0;i++)
    {
        if((*Fsimp)[i]==0)
        {T0[k]=i;
        k++;
        }
    }
    T0[0]=k-1;

```

```

k=2;j=1;
for(i=1;i<=T0[0];i++)
{
    if(T0[i]!=(*Top1)[k])
        {(*Top0)[j]=T0[i];j++;}
    else
        {
            if(k<(*Top1)[0])
                k++;
        }
}
(*Top0)[0]=j-1;
return 1;
}
#include "graph.h"
int aaaaaaac(void *);
int aaaaaaad(void *);
int aaaaaaae(void *);
int saaaaaab(void *);
int aaaaaaah(void *);
int aaaaaaaj(void *);
int aaaaaaak(void *);
int saaaaaaa(void *);
int aaaaaaal(void *);
int aaaaaaam(void *);
int aaaaaaan(void *);
int aaaaaaaao(void *);
int gaaaaaab(void *);
int paaaaaaa(void *);
int paaaaaab(void *);
int paaaaaac(void *);
int paaaaaad(void *); int GraphMV(void *,int,int,DEFPRED*,DEFTOP*,DEFGRAF*);
static DEFPRED ListPred[4]=
{
    {"paaaaaaa", paaaaaaa},
    {"paaaaaab", paaaaaab},
    {"paaaaaac", paaaaaac},
    {"paaaaaad", paaaaaad}
};
static DEFTOP ListTop[15]=
{
    {"aaaaaac",0, 0, &aaaaaac},
    {"aaaaaad",1, 1, &aaaaaad},
    {"aaaaaaae",2, 2, &aaaaaaae},
    {"aaaaaaaf",3, 3, &saaaaaab},
    {"aaaaaaah",4, 4, &aaaaaaah},
    {"aaaaaaai",5, 5, &saaaaaab},
    {"aaaaaaj",6, 6, &aaaaaaj},
    {"aaaaaak",7, 8, &aaaaaak},
    {"aaaaaaag",9, 9, &saaaaaaa},
    {"aaaaaal",10, 10, &aaaaaal},
    {"aaaaaam",11, 11, &aaaaaam},
    {"aaaaaan",12, 13, &aaaaaan},
    {"aaaaaak",14, 15, &aaaaaak},
    {"aaaaaaao",16, 16, &aaaaaaao},
    {"gaaaaaab",-77, -77, &gaaaaaab}
};

```

```

static DEFGRAF ListGraf[17]=
{
    {0, 1 },
    {0, 2 },
    {0, 3 },
    {0, 4 },
    {0, 5 },
    {0, 6 },
    {0, 7 },
    {1, 4 },
    {0, 8 },
    {0, 9 },
    {0, 10 },
    {2, 11 },
    {3, 13 },
    {0, 12 },
    {2, 11 },
    {0, 13 },
    {0, 14 }
};
int gaaaaaaa(void *p[])
{ int Ntop =15;
int jroot = 0;

    GraphMV(p,jroot,Ntop,ListPred,ListTop,ListGraf);
    return(1);
}

```